

# Cumhuriyet Üniversitesi

## İşletim Sistemleri Dersi

### Bölüm 8

### *Sanal Bellek*

### *(Virtual Memory)*

Dr. Halil ARSLAN

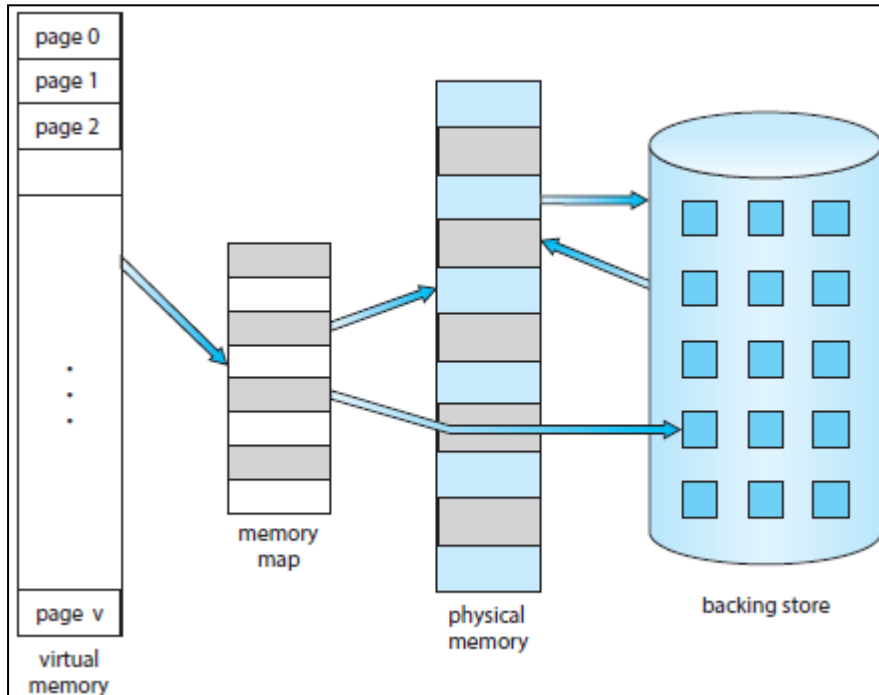
# Bölümün hedefleri

- Sanal Bellek Konsepti - Background
- Sayfalama İsteği - Demand paging
- Kopyala ve Yaz - Copy and write
- Sayfa Değişimi - Page Replacement
- Çerçeve Tahsisi - Allocation of Frames
- Çökme - Thrashing
- Bellek Sıkıştırma - Memory Compression
- Çekirdek Bellek Tahsisi - Allocating Kernel Memory
- Diğer konular - Prepaging, Page Size, TLB, IPT
- İşletim Sistemi Örnekleri - Linux, Windows

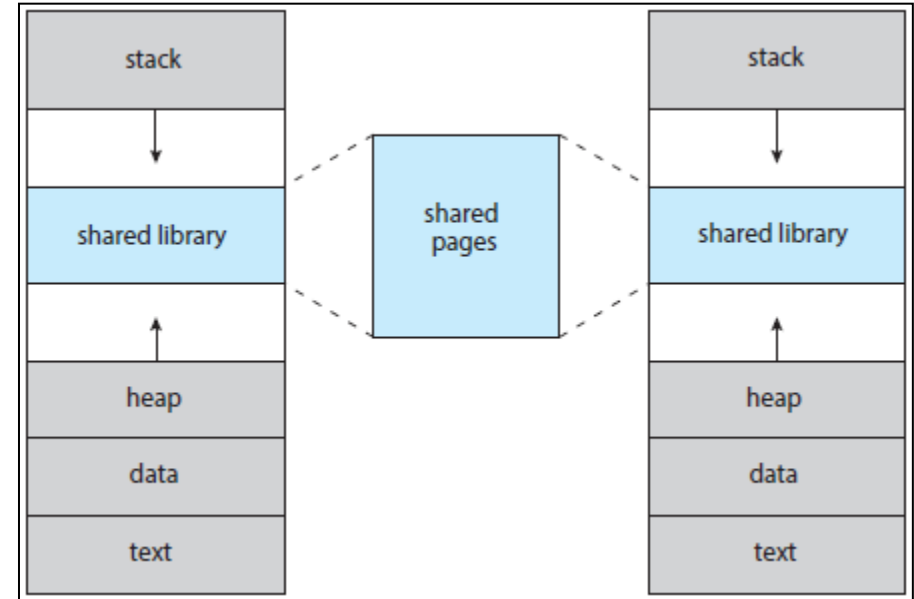
# Sanal Bellek - (Virtual Memory)

- Bilgisayar sistemlerinde kullanılan çeşitli bellek yönetim stratejileri, birçok süreci aynı anda bellekte tutmayı (multiprogramming) içerir.
  - Ancak, bir sürecin yürütülmeden önce tümüyle bellekte olması gereksinimi vardır.
- Sanal bellek, süreçlerin tamamen bellekte olmadan yürütülmesini sağlayan bir tekniktir.
  - Bu tekniğin başka bir avantajı da aktif programların fiziksel bellekten daha büyük olabilmesidir.
- Ayrıca sanal bellek, programcı gözünde mantıksal belleği soyutlayarak fiziksel bellekten ayırır ve ana belleği tek tip bir depolama dizesi şeklinde sunar.
  - Bu teknik, programcıların bellek depolama sınırlamalarından bağımsız hareket etmesini sağlar.
- Sanal bellek, süreçlerin dosyaları ve kütüphaneleri paylaşmasına imkan sunar.
- Ek olarak, süreç oluşturma için verimli mekanizmalar sağlar.
- Ancak, sanal belleğin uygulanması zordur ve dikkatsizce kullanılırsa performansı önemli ölçüde düşürebilir.

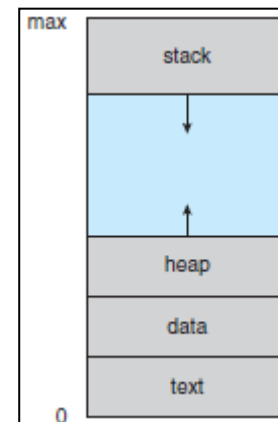
# Sanal Bellek - (Virtual Memory)



**Sanal bellek, fiziksel bellekten daha büyük olabilir**



**Paylaşılan kütüphanelerin sanal bellek yerleşimi**



**Process sanal adres uzayı**

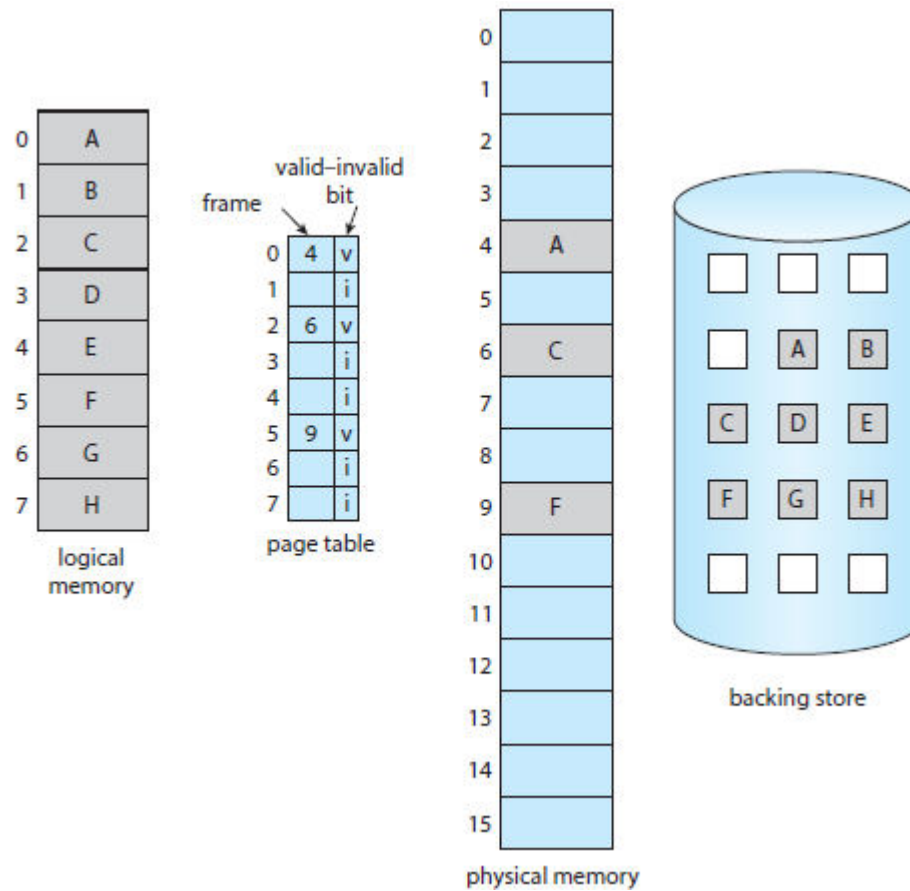
# Sayfalama İsteği - Demand paging

- Çalıştırılabilir bir programın yardımcı depolama birimlerinden (Örn. HDD) belleğe yüklenmesi sürecinde;
  - Tüm program, yürütme zamanında (execution-time) fiziksel belleğe yüklenebilir.
    - Ancak bu yaklaşımdaki temel sorun; başlangıçta tüm programın bellekte olmasına ihtiyaç duyulmayabilir.
  - Alternatif yaklaşım, sayfaları yalnızca ihtiyaç duyuldukça belleğe yüklemektir.
    - Bu teknik, genellikle sanal bellek sistemlerinde kullanılır ve **Sayfalama İsteği (Demand Paging)** olarak bilinir.
    - Sayfalama istekli sanal bellek (demand-paged virtual memory) tekniği ile sayfalar, program yürütülürken talep edildiğinde fiziksel belleğe yüklenir.
      - Erişilmeyen sayfalar fiziksel belleğe yüklenmez.
    - Sayfalama İsteği yaklaşımı, süreçlerin ikincil bellekte (genellikle bir HDD) bulunduğu, takaslamalı (swapping) bir sayfalama sistemine benzer.
    - Bu yaklaşım, sanal belleğin birincil avantajı olan yalnızca gereken program bölümlerini fiziksel belleğe yükleyerek belleğin verimli kullanımını açıklar.

# Demand paging – Temel konsept

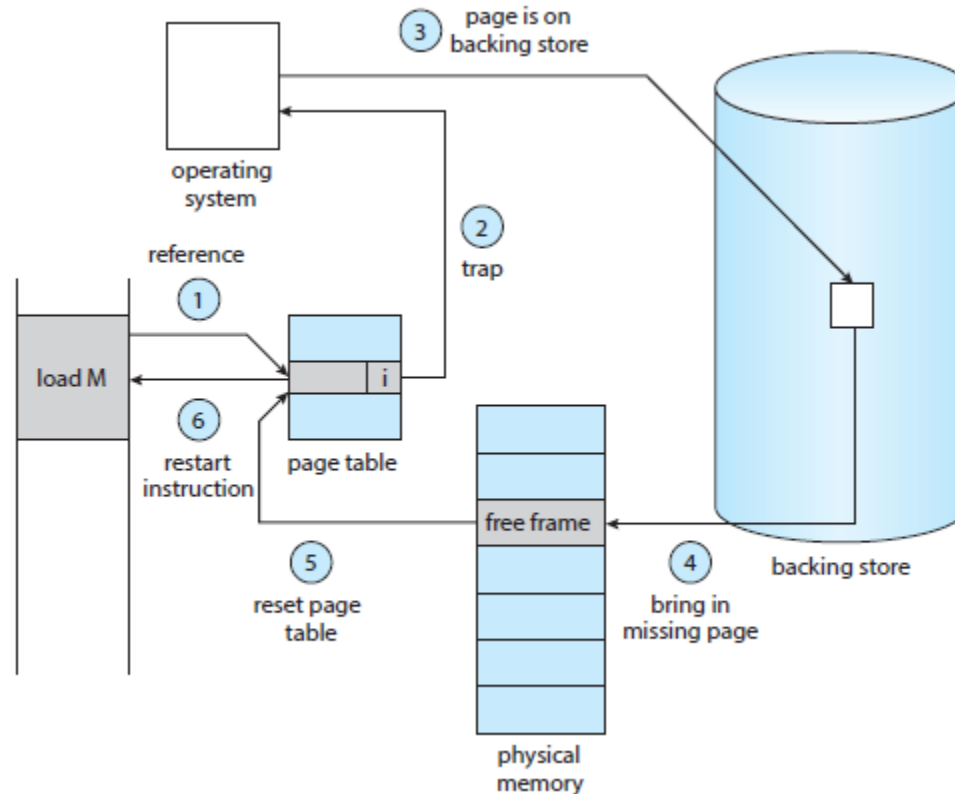
- Sayfalama İsteği tekniğinin arkasındaki temel konsept, bir sayfayı fiziksel belleğe yalnızca ihtiyaç duyulduğunda yüklemektir.
  - Sonuç olarak, bir süreç yürütülürken bazı sayfaları bellekte, bazı sayfaları ise ikincil depolama biriminde olacaktır.
- Bu nedenle, bu yapıları yönetmek için donanım desteğine ihtiyaç vardır.
- Örneğin page-table'da yer alan valid-invalid bit'leri bu amaçla kullanılır.
  - Bit «valid» olarak ayarlandığında, ilgili page hem sanal bellekte geçerli bir page'dir hem de fiziksel bellekte'dir.
  - Bit «invalid» olarak ayarlanmışsa, page ya geçerli değildir (yani, sürecin mantıksal adres alanında değildir) yada şu anda ikincil depolama birimindedir.
    - Ana Belleğe yüklenen bir sayfanın page-table kaydı «valid» olarak ayarlanır.
    - Ana Bellekte olmayan bir sayfanın page-table kaydı «invalid» olarak ayarlanır.
  - Bir süreç, invalid olarak işaretlenen sayfaya çalışma-süresi boyunca erişmezse, bu sayfanın invalid olarak işaretlenmesinin hiçbir etkisi olmayacaktır.

# Demand paging – Temel Konsept



# Demand paging – Temel konsept

- Bir süreç, belleğe yüklenmemiş bir sayfaya erişmeye çalışırsa **sayfa hatası (page fault)** oluşur.
  - MMU, mantıksal adresi page-table aracılığıyla fiziksel adrese çevirirken, invalid biti görür (trap oluşması).





# Demand paging – Temel konsept

- Hafızada hiçbir sayfası (page) olmayan bir süreç yürütülmeye başlanabilir.
  - İşletim sistemi, komut pointer'ını (IP) bellekte olmayan bir sayfada bulunan ilk komuta ayarladığında, **page fault** verir.
    - Bu sayfa hafızaya yüklendikten sonra sürecin ihtiyaç duyduğu diğer sayfa'lar page fault vererek hafızaya getirilecek ve süreç yürütülmeye devam edecektir.
  - Belli bir noktadan sonra artık hata oluşmadan süreç çalışabilecektir.
- Bu teknik, **Yalın Sayfalama İsteği (pure demand paging)** olarak ifade edilir.
  - Gerekli olana kadar hiçbir sayfayı belleğe yükleme!
- Teorik olarak bazı programlar, her komut yürütme durumunda birden çok yeni bellek sayfasına erişebilir (komut için bir sayfa ve veri için birçok sayfa).
  - Böylece komut başına birden çok sayfa hatası ortaya çıkabilir.
  - Bu durum, sistem performansı açısından kabul edilemezdir.
  - Yapılan program analizleri, bu durumun çok fazla olmadığını göstermektedir.
    - Sonraki bölümlerde açıklandığı gibi programlar, **referans yerelliğine (locality of reference)** sahip olma eğilimindedirler ve bu da Sayfalama İsteğinde makul performans sağlar.

# Demand paging – Temel konsept

- Demand paging donanımı ile paging ve swapping donanımları aynıdır.
  - **Sayfa tablosu (page-table):** Bu tablo, valid-invalid bitleri veya koruma bitlerinin özel değerleriyle geçersiz bir kaydı işaretler.
  - **İkincil bellek:** Bu hafıza, ana hafızada bulunmayan sayfaları tutar. İkincil bellek genellikle yüksek hızlı bir disk yada NVM'dir. Takas cihazı olarak da bilinir. Bu amaçla kullanılan bölüme **swap space** denir.
- Sayfalama İsteği için en önemli gereksinim, bir page-fault'tan sonra ilgili komutun yeniden işletilebilinmesidir.
  - Sayfa hatası oluştuğunda kesintiye uğrayan sürecin durumu (PCB) saklandığı için, süreç tam olarak aynı yerde ve durumda yeniden başlatılabilir.
    - Çünkü istenen sayfa artık bellekte ve erişilebilirdir. Çoğu durumda, bu gereksinimin karşılanması kolaydır.
    - Herhangi bir bellek referansında bir sayfa hatası meydana gelebilir. Komut getirme durumunda sayfa hatası meydana gelirse, komut yeniden işletilir.
    - Bir operantı alırken bir sayfa hatası meydana gelirse, komut tekrar çağırılmalı, kodu çözmeli ve ardından operant getirilmelidir.

# Demand paging – Temel konsept

- En kötü durum için bir örnek olması bakımından;
  - A ile B'yi toplayıp sonucu C'de saklayan üç adresli bir komut düşünelim.
    1. Komutu (Add) al ve kodunu çöz. (Fetch - decode)
    2. A'yı getir. (Fetch A)
    3. B'yi getir. (Fetch B)
    4. A ile B'yi topla. (Add)
    5. Toplamı C'ye sakla. (Store)
- Sayfa hatası, toplamı C'ye kaydetmeye çalışıldığında ortaya çıkarsa (C'nin bellekte olmayan bir sayfada olduğunu varsayalım), ilgili sayfa diskten alınmalı, getirilmeli, page-table güncellenmeli ve komut yeniden çalıştırılmalıdır.
  - Yeniden çalıştırma durumunda, komutun yeniden getirilmesi, kodunun tekrar çözülmesi, iki operantın tekrar getirilmesi ve tekrar toplama yapmayı gerektirecektir.
  - Ancak, bilinmelidir ki bu tekrarlamalar yalnızca bir sayfa hatası ortaya çıktığında gereklidir.

# Demand paging – Free-Frame List

- Bir sayfa hatası oluştuğunda, işletim sistemi istenen sayfayı ikincil depolama biriminden ana belleğe getirmelidir.
  - Çoğu işletim sistemi **free-frame list** adında bir boş çerçeve havuzu tutar.
    - Bir süreçteki stack veya heap segmentler genişlediğinde de çerçeve tahsis edilmelidir.
- İşletim sistemleri, **zero-fill-on-demand** olarak bilinen bir teknik kullanarak boş çerçeveler tahsis eder.
  - Çerçeveler tahsis edilmeden önce "sıfırlanır" ve böylece önceki içerikler silinir.
- Bir sistem başladığında, tüm kullanılabilir bellek serbest çerçeve listesine alınır.
  - Serbest çerçeveler istendikçe, serbest çerçeve listesinin (free-frame list) boyutu küçülür.
    - Bir noktada, liste ya sıfıra düşer ya da belirli bir eşiğin altına düşer. Bu durumda listenin yeniden düzenlenmesi (**page replacement** konusu) gerekir.



# Demand paging – Performans

- Sayfalama İsteği, bilgisayar sisteminin performansını önemli ölçüde etkileyebilir. Örneğin;
  - Bellek erişim süresinin 10 nanosaniye olduğunu varsayalım. (memory access,  $Ma$  )
  - Sayfa hatası olmadığı sürece, etkin erişim süresi  $Ma$ 'ya eşittir.
  - Bir sayfa hatası olursa, önce ilgili sayfa ikincil depolamadan okunmalı ve ardından istenen veriye erişilmelidir.
- $P$ , bir sayfa hatası olasılığı olsun ( $0 \leq P \leq 1$ ).
  - $P$ 'nin sıfıra yakın olması istenir.
  - Yani, sadece birkaç sayfa hatası olması istenir.
- Bu durumda Etkin Erişim Süresi;
  - Etkin erişim süresi =  $(1 - p) \times Ma + p \times \text{sayfa hata süresi}$ .
- Etkin erişim süresini hesaplayabilmek için bir sayfa hatası/düzeltilme durumunda ne kadar zaman gerektiğini bilmek gerekir.
  - Bir sayfa hatası, belirli işlemlerin oluşmasına neden olur.

# Demand paging – Performans

1. İşletim sisteminde trap oluşur.
2. Registerlar ve süreç durumu saklanır.
3. Kesmenin bir sayfa hatasından olduğu belirlenir.
4. Sayfa referansının valid/invalid kontrolü ve sayfanın ikincil depolama konumu belirlenir.
5. Disk'ten boş çerçeveye okuma yapılır
  - a. Okuma talebi karşılanıncaya kadar kuyrukta beklenir.
  - b. Disk arama ve/veya gecikme süresi beklenir.
  - c. Sayfa boş çerçeveye aktarılır.
6. Bekleme durumunda, CPU başka bir sürece tahsis edilir.
7. Depolama biriminden kesme alınır (G/Ç tamamlandı).
8. Diğer süreç için registerlar ve süreç durumu saklanır (adım 6 gerçekleşmişse).
9. Kesmenin ikincil depolama cihazından geldiği belirlenir.
10. İstenilen sayfanın bellekte olduğunu göstermek için page-table güncellenir.
11. CPU'nun bu sürece yeniden tahsis edilmesi beklenir.
12. Registerlar, süreç durumu ve yeni page-table geri yüklenir ve kesilen komut devam eder.

# Demand paging – Performans

- Her durumda, sayfa hatası servisinin üç ana görevi vardır:
  1. Sayfa hatası kesme servisi
  2. Sayfa'nın okunması.
  3. Sürecin yeniden başlatılması
- Birinci ve üçüncü görevlerin her biri 1 ila 100 mikrosaniye sürebilir.
- İkinci görev, HDD performansına bağlı olarak 8 milisaniyeye yakın olabilir.
  - Tipik bir sabit diskin ortalama gecikme süresi 3 milisaniye, arama süresi 5 milisaniye ve aktarım süresi 0,05 milisaniye'dir.
- Ortalama sayfa hatası servis süresi 8 milisaniye, bellek erişim süresi 200 nanosaniye ile, etkin erişim süresi

$$\begin{aligned}\text{etkin erişim süresi} &= (1 - p) \times (200) + (p \times (8 \text{ milisaniye})) \\ &= (1 - p) \times 200 + p \times 8.000.000 \\ &= 200 + 7.999.800 \times p.\end{aligned}$$

# Demand paging – Performans

- O halde, etkin erişim süresinin, sayfa hatası oranıyla doğru orantılı olduğu görülmektedir.
- Her 1000 erişimde bir sayfa hatası oluşursa, etkin erişim süresi 8,2 mikrosaniye olur.
  - Sayfalama İsteği nedeniyle bilgisayar 40 kat daha yavaşlayacaktır!
- Performans düşüşünün yüzde 10'dan az olmasını istiyorsak, sayfa hatası olasılığını aşağıdaki seviyede tutmamız gerekir:  $200 \% 10 \Rightarrow 220 \text{ ns'den az!}$ 

$$220 > 200 + 7,999,800 \times p,$$

$$20 > 7.999.800 \times p,$$

$$p < 0.0000025.$$
- Yani, sayfalamadan kaynaklanan yavaşlamayı makul bir seviyede tutmak için 399.990 bellek erişiminin birinden daha azında sayfa hatası oluşmalıdır.
  - Özetle demand paging sisteminde sayfa hatası oranını düşük tutmak önemlidir.
    - Aksi takdirde etkin bellek erişim süresi artar ve sürecin yürütümü önemli oranda yavaşlar.



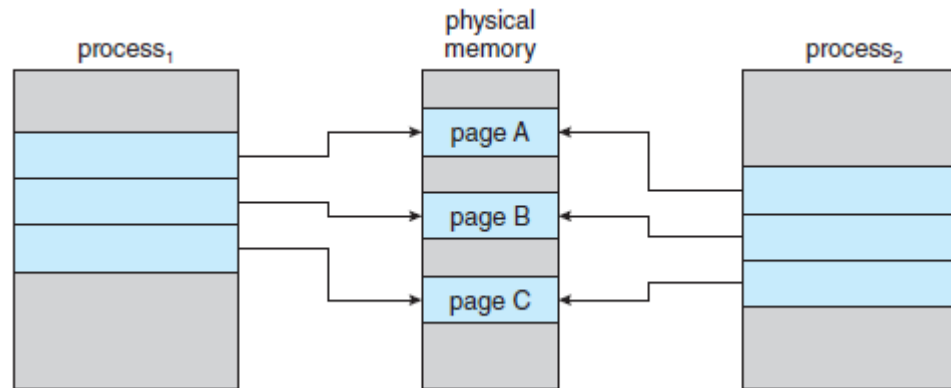
# Demand paging – Performans

- Sayfalama İsteğinin önemli bir yönü swap alanının kullanılmasıdır.
  - Genellikle swap alanındaki G/Ç, dosya sistemine göre daha hızlıdır.
- Sistemin daha iyi sayfalama verimi sunması için bir süreç başlangıçta dosya imajını swap alanına kopyalar ve ardından swap alanından demand paging'i gerçekleştirir.
  - Bu yaklaşımın dezavantajı, başlangıçta programın dosya imajını kopyalamasıdır.
- Diğer bir seçenek olarak (Linux ve Windows) başlangıçta dosya sisteminden Demand Paging yapmak ve belleğe taşındıkça swap alanına sayfaları atmaktır.
  - Böylece, gerekli sayfalar dosya sisteminden, sonraki sayfalar swap alanından okunur.
- Bazı sistemler, çalıştırılabilir binary dosyaların swap kullanımını sınırlandırır.
  - Bu tür dosyalar için Demand Paging doğrudan dosya sisteminde yürütülür. (Linux ve BSD)
  - Ancak, bir dosyayla ilişkili olmayan sayfalar (**anonymous memory**) swap alanı kullanır (bir süreç için heap ve stack içeriği).
- Mobil işletim sistemleri swapping'i desteklemez.
  - Bu sistemler dosya sisteminden sayfa talep eder.

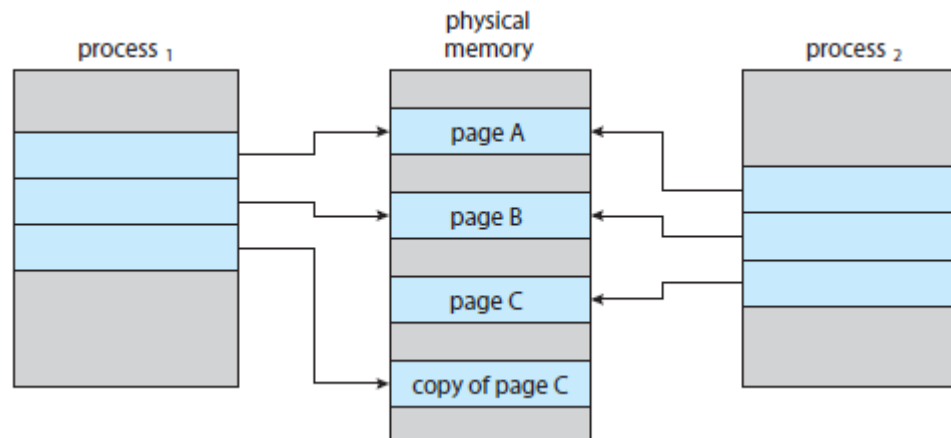
# Copy-on-Write – Yazarken Kopyala

- `fork()` sistem çağrısını kullanarak süreç oluşturma, sayfa paylaşımına benzer bir teknik kullandığı için başlangıçta demand paging ihtiyacı oluşmaz.
- Bu teknik, hızlı işlem oluşturma sağlar ve yeni oluşturulan sürece atanması gereken yeni sayfaların sayısını en aza indirir.
- `Fork()` sistem çağrısıyla parent sürecin kopyası bir child süreç oluşturulduğu için ebeveynin adres alanının bir kopyası oluşturulur.
  - Ebeveyne ait sayfalar kopyalanarak çalıştırılır
  - Ancak, birçok child süreç oluşturulduktan hemen sonra `exec()` sistem çağrısı kullandığı için parent sürecin adres alanının kopyalanması gereksiz olabilir.
- Bunun yerine, parent ve child süreçlerin başlangıçta aynı sayfaları paylaşmasına izin vererek çalışan, Copy-on-Write olarak bilinen bir teknik kullanılabilir.
  - Paylaşılan sayfalar, Copy-on-Write sayfaları olarak işaretlenir.
  - Her iki süreçte paylaşılan bir sayfaya yazarsa, paylaşılan sayfanın bir kopyası oluşturulur.

# Copy-on-Write – Yazarken Kopyala



**Process-1 Page-C'yi değiştirmeden önce**



**Process-1 Page-C'yi değiştirdikten sonra**

# Copy-on-Write – Yazarken Kopyala

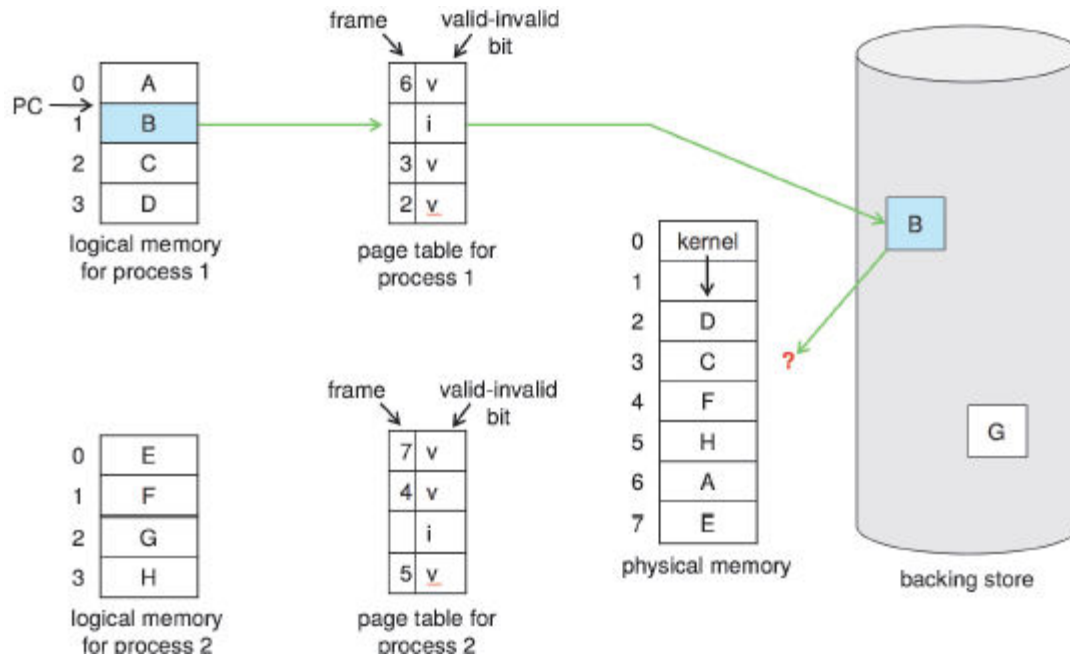
- Örneğin, child süreç copy-on-write olarak işaretlenmiş ve stack'in bölümlerini içeren bir sayfayı değiştirmeye çalışsın.
- İşletim sistemi, free-frame-list'ten bir çerçeve alarak bu sayfanın bir kopyasını oluşturur ve o sayfayı child sürecin adres alanına eşler.
  - Child süreç daha sonra kopyalanan sayfayı değiştirecek.
  - Bu teknik, Windows, Linux ve macOS gibi birçok işletim sistemi tarafından kullanılır.
- UNIX'in çeşitli sürümleri (Linux, macOS ve BSD Unix) **vfork()** olarak bilinen fork() sistem çağrısının bir varyantını sağlar.
  - **vfork()** ile parent süreç askıya alınır ve child süreç ebeveynin adres uzayını kullanır.
  - vfork() copy-on-write kullanmadığından, child süreç ebeveynin adres alanının herhangi bir sayfasını değiştirirse, değiştirilen sayfalar parent süreç tarafını etkiler.
    - vfork() ile türetilen child süreç, ebeveynin adres alanını değiştirebileceği için dikkatlice kullanılmalıdır.
  - vfork(), child süreç oluşturulduktan hemen sonra exec() çağrısını kullanmak üzere tasarlanmıştır. Sayfalar kopyalanmadığından son derece verimlidir.

# Page Replacement – Sayfa Değişimi

- Bir sayfa ilk kez referans edildiğinde en fazla bir kez sayfa hatası oluşacağı teorik olarak varsayılsa da bu durum gerçekte doğru değildir.
  - Örn, 10 sayfalık bir süreç, sayfalarının yarısını kullanıyorsa, Sayfalama İsteği hiç kullanılmayan 5 sayfayı yüklemek için gerekli G/Ç'tan tasarruf eder.
  - Böylece, iki kat daha fazla süreç çalıştırılarak multi-programming yeteneği artırılabilir.
- Yada, 40 boş çerçeve olsaydı, her biri 10 çerçeve kullanan 4 süreç yerine (5'i hiç kullanılmadığı için) 8 süreç çalıştırılabilirdi.
  - Multi-programming kapasitesi artırılarak hafıza aşırı tahsis edilmiş olur.
  - Her biri 10 sayfa boyutunda olan ancak yalnızca 5 sayfa kullanan 6 süreç çalıştırılırsa, 10 çerçeve yedekte kalırdı.
  - Ancak, bu süreçlerin hepsi bir anda 10 sayfasının tümünü kullanmayı isterse, toplam 40 çerçeve olduğu için fazladan 20 çerçeveye ihtiyaç duyulurdu.
  - Ayrıca sistem belleğinin tümü süreçlere tahsis edilmez. (G/Ç tamponları vs.)
    - G/Ç tamponları ve Programlanabilir hafızası tahsisi oransal yada rekabete dayalı olabilir.

# Page Replacement – Sayfa Değişimi

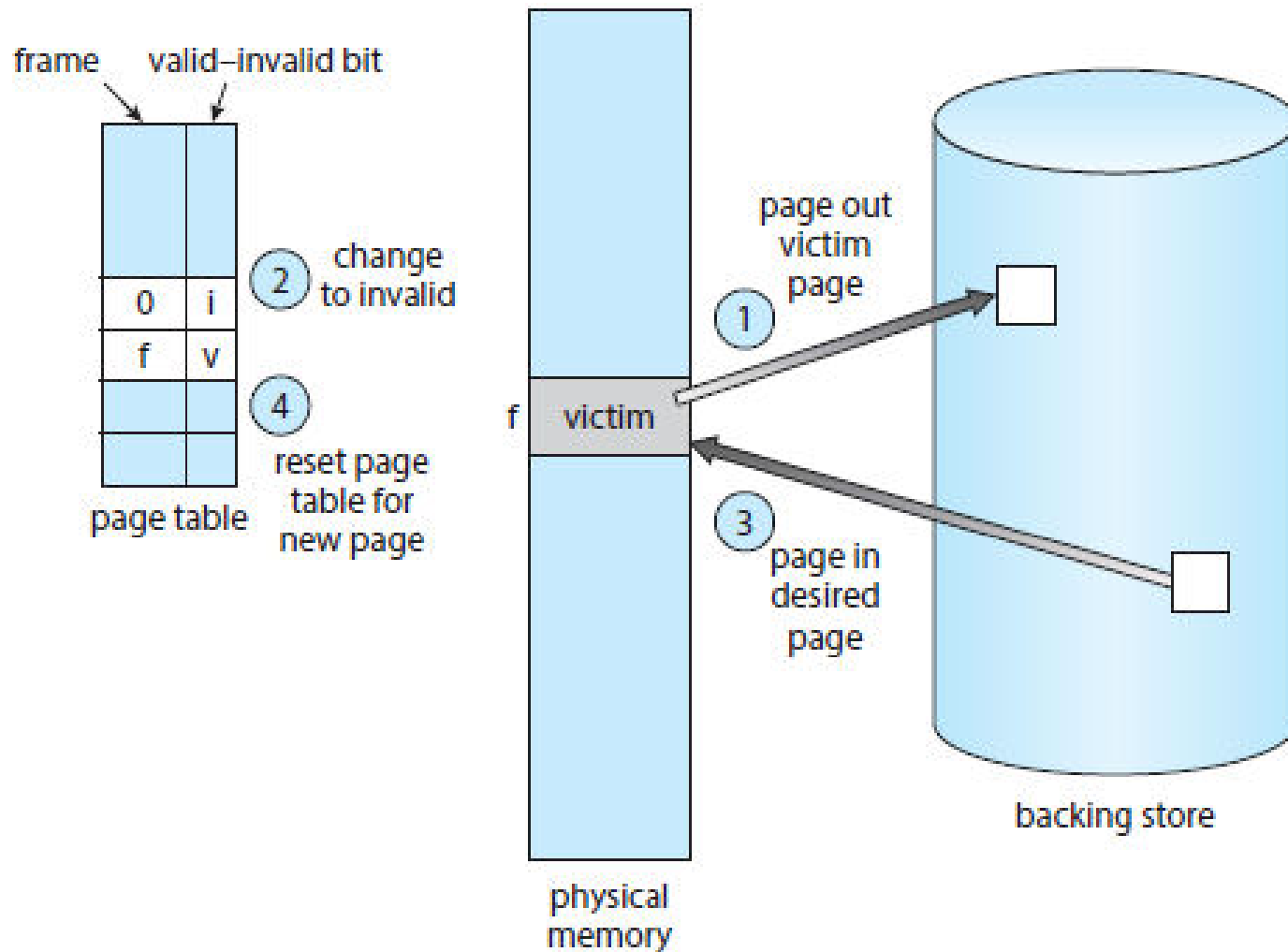
- Belleğin aşırı tahsisinde, bir süreç yürütülürken bir sayfa hatası oluşursa;
  - İşletim sistemi, istenen sayfanın yardımcı depolamada nerede bulunduğunu belirler,
  - Ancak serbest çerçeve listesinde boş çerçeve olmadığı için İşletim sistemi;
    - Süreci sonlandırabilir (iyi bir seçenek değil),
    - Standart takaslamayı (swapping) kullanabilir, ek yük nedeniyle çoğu OS desteklemez.
    - **Sayfa Değişimi (Page Replacement)** ile takaslanan sayfaları okuyabilir. (Uygun)



# Page Replacement – Sayfa Değişimi

- **Sayfa Değişimi** yaklaşımının temeli;
  - Boşta hiçbir çerçeve yoksa, aktif olarak kullanılmayan bir çerçeve bulunur ve o çerçeve serbest (free) hale getirilir.
  - Bir çerçeve, içeriği takas-alanına yazılarak ve page-table'ı artık o çerçevenin bellekte olmadığı işaretlenerek (invalid) serbest (free) bırakılır.
  - Böylece sürecin hatalı sayfasını tutmak için bu serbest çerçeve kullanılır.
- Sayfa hatası servis rutinini Sayfa Değişimini içerecek şekilde güncellersek:
  1. İkincil depolama biriminden istenen sayfanın konumunu bul.
  2. Boş çerçeve bul:
    - a. Boş çerçeve varsa, onu kullan.
    - b. Boş çerçeve yoksa, hedef çerçeveyi seçmek için bir sayfa değişimi algoritması kullan.
    - c. Hedef çerçeveyi ikincil depoya sakla (gerekirse); page ve frame tablolarını güncelle.
  3. İstenen sayfayı serbest çerçeveye yükle; sayfa ve çerçeve tablolarını güncelle.
  4. Sayfa hatasının olduğu yerden sürece devam et.

# Page Replacement – Sayfa Değişimi





# Page Replacement – Sayfa Değişimi

- Hiçbir çerçeve boş değilse, iki sayfa aktarımı (page in/out) gerekmektedir.
  - Bu durum, sayfa hatası servis süresini ikiye katlar ve etkin erişim süresini artırır.
- Bu ek yük bir **modifiye bit (veya dirty bit)** kullanılarak azaltılabilir.
  - Her sayfa veya çerçeve, kendisiyle ilişkilendirilmiş bir modifiye bitine sahiptir.
  - Modifiye biti, sayfadaki herhangi bir byte yazıldığında donanım tarafından ayarlanır.
  - Sayfa Değişimi için bir sayfa seçildiğinde, modifiye biti incelenir.
    - Bit set ise, sayfanın ikincil depolama biriminden okunduktan sonra değiştirildiği anlaşılır.
      - Bu durumda sayfa tekrar ikincil depolama birimine yazılmalıdır.
    - Ancak modifiye biti ayarlanmamışsa, sayfa değiştirilmemiştir.
      - Bu durumda, bellek sayfasını ikincil depolama birimine yazmak gerekmez. Zaten orijinal halidir.
  - Bu teknik, salt okunur sayfalar için de geçerlidir (örneğin, binary code pages).
    - Bu tür sayfalar değiştirilemez; Bu nedenle, istendiğinde bellekten atılabilirler.
    - Bu şema, sayfa değiştirilmediyse G/Ç süresini yarı yarıya azaltır ve sayfa hatası servis süresini önemli ölçüde azaltabilir.

# Page Replacement – Sayfa Değişimi

- Sayfa Değişimi (**Page Replacement**), Sayfalama İsteğinin (**Demand Paging**) temelidir.
  - Bu makinizmalar ile programcılar için daha küçük bir fiziksel bellekte muazzam bir sanal bellek sağlayarak mantıksal bellek ile fiziksel bellek arasında ayırım sağlanır.
  - 20 sayfalık bir süreç, Sayfalama İsteği ve gerektiğinde boş bir çerçeve bulmak için bir Sayfa Değişimi algoritması kullanarak 10 çerçeve ile yürütülebilir.
  - Modifiye edilmiş bir sayfa için Sayfa Değişimi yapılacaksa, içeriği ikincil depolama birimine kopyalanır.
- Sayfalama İsteğini uygulamak için iki büyük sorunu çözmeliyiz:
  - **Çerçeve ayırma (frame-allocation) algoritması,**
  - **Sayfa Değişim (page-replacement) algoritması**
    - Bellekte birden fazla süreç varsa, her bir sürece kaç çerçeve tahsis edeceğine karar verilmeli
    - Sayfa Değişimi gerektiğinde, değiştirilecek çerçeveleri seçmek gerekli.
      - Uygun algoritma tasarlamak önemlidir, çünkü ikincil depolama birimi Giriş/Çıkış işlemleri çok maliyetlidir.

# Page Replacement – Sayfa Değişimi

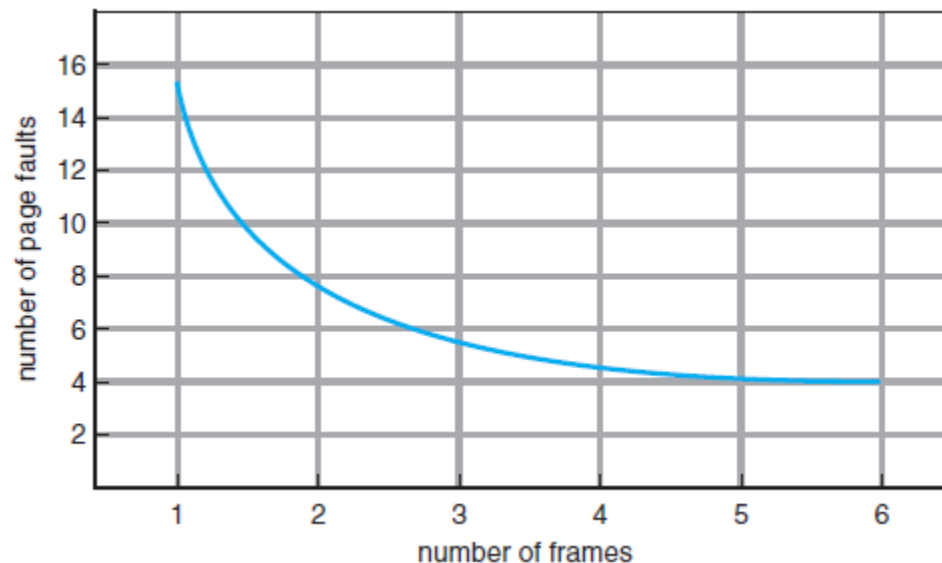
- Birçok farklı Sayfa Değişim algoritması vardır.
  - Algoritma seçiminde, en düşük sayfa hatası oranına (**page-fault rate**) bakılır.
  - Bir algoritma bir bellek referans dizisinde (**reference string**) çalıştırılır ve sayfa hatası oranı hesaplanır. Referans Dizeleri iki şekilde üretilebilir;
    - (1)Yapay referans dizeleri (random), (2) Belirli bir sistem izlenerek.
    - İkinci seçenek çok sayıda veri üretir (1 milyon adres/sn). Veri sayısı azaltılabilir.
      - Belirli bir page-size için, adresin tamamı yerine yalnızca sayfa numarası dikkate alınır.
      - Bir p sayfasına referans varsa, p sayfasına yapılan hemen sonraki referans asla bir sayfa hatasına neden olmaz. Çünkü p sayfası ilk referanstan sonra bellekte olacaktır.
    - Örneğin bir süreci belirli bir süre izlersek:
 

0100, 0432, 0101, 0612, 0102, 0103, 0104, 0101, 0611, 0102, 0103, 0104, 0101, 0610, 0102, 0103, 0104, 0101, 0609, 0102, 0105

gibi bir veri elde edebiliriz.
  - Page-size 100 byte ise; referans dizisi: 1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1 şeklinde indirgenir.

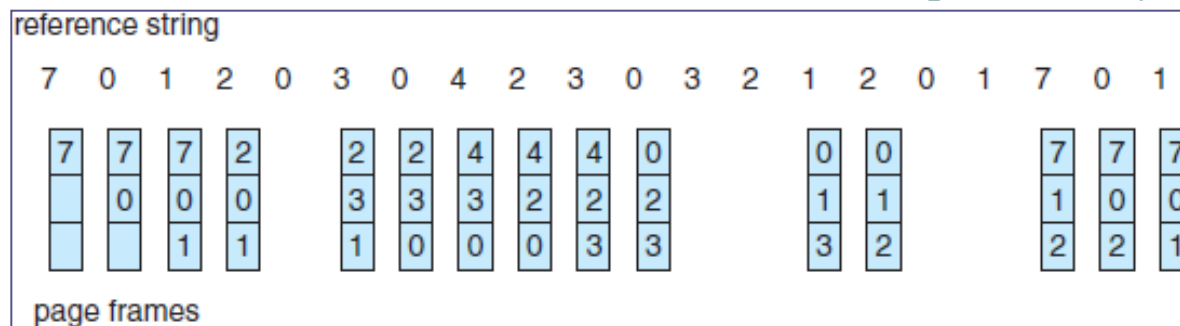
# Page Replacement – Sayfa Değişimi

- Belirli bir Referans Dizesi ve Sayfa Değişimi algoritması için sayfa hatalarının sayısını belirlemek için, mevcut çerçeve sayısını da bilmek gerekir.
  - Mevcut çerçeve sayısı arttıkça, sayfa hatası sayısı azalır.
    - Örneğin önceki referans dizesi için, üç veya daha fazla çerçeve olsaydı, yalnızca üç sayfa hatası olurdu (her sayfanın ilk referansı için bir hata).
    - Ancak, bir çerçeve olsaydı, her referans bir hataya neden olurdu (11).
  - Sonraki bölümlerde 3 çerçeveli bir bellek için aşağıdaki referans dizesi kullanılacak  
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1



# Page Replacement – FIFO

- FIFO (ilk-giren ilk-çıkart) her sayfayla o sayfanın belleğe varış zamanını ilişkilendirir.
  - Bir sayfanın değiştirilmesi gerektiğinde, en eski sayfa seçilir.
    - Bir sayfanın belleğe getirildiği zamanı saklamaya gerek yok!
  - Belleğe getirilen tüm sayfalar bir FIFO kuyruğunda tutulur. Sıranın başındaki sayfa seçilir. Bir sayfa belleğe getirildiğinde, o sayfa kuyruğun sonuna yerleştirilir.
- Örnek referans dizesi için, üç çerçeve başlangıçta boştur.
  - İlk üç referans (7, 0, 1) sayfa hatalarına neden olur ve boş çerçevelere yerleştirilir.
  - Sonraki referans P2, ilk sırada olan P7'nin yerine,
  - Sonraki referans P0, sayfa hatasına neden olmaz (P0 zaten bellekte).
  - Sonraki referans P3, ilk sırada olan P0'ın ... şeklinde toplam 15 sayfa hatası oluşur.

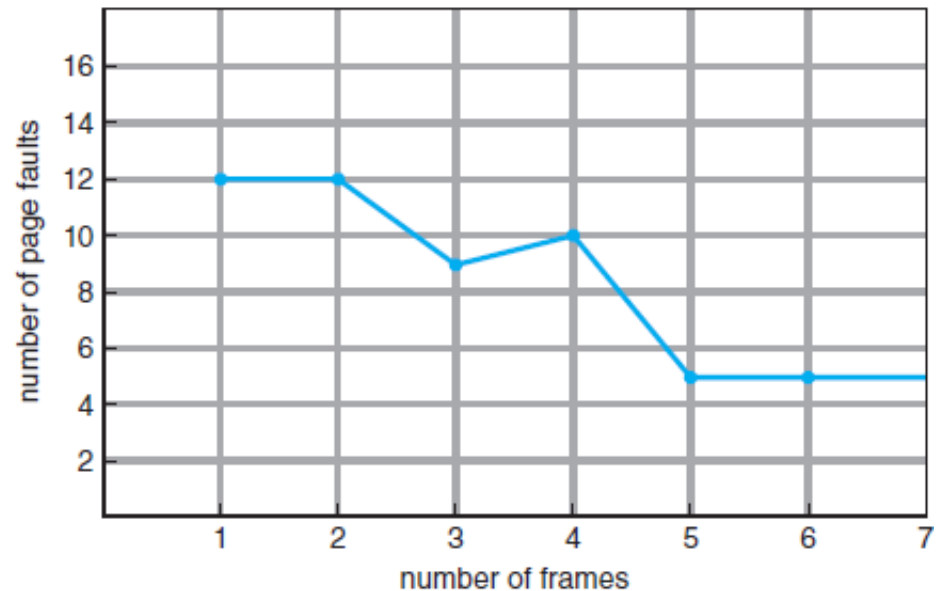


# Page Replacement – FIFO

- FIFO Sayfa Değiştirme algoritmasının performansı iyi değildir.
  - Değiştirilen sayfa, önceleri yüklenen ancak sürekli kullanılan bir değişken içerebilir.
  - Ayrıca değişim için aktif kullanımda olan bir sayfa seçilebilir.
    - Dolayısıyla aktif sayfa tekrar referans edildiğinde yeniden bir hata oluşur.
  - Bu nedenle, kötü bir seçim, sayfa hata oranını artırır ve süreci yavaşlatır.
- FIFO Sayfa Değiştirme algoritmasının sorunlarını göstermek için:

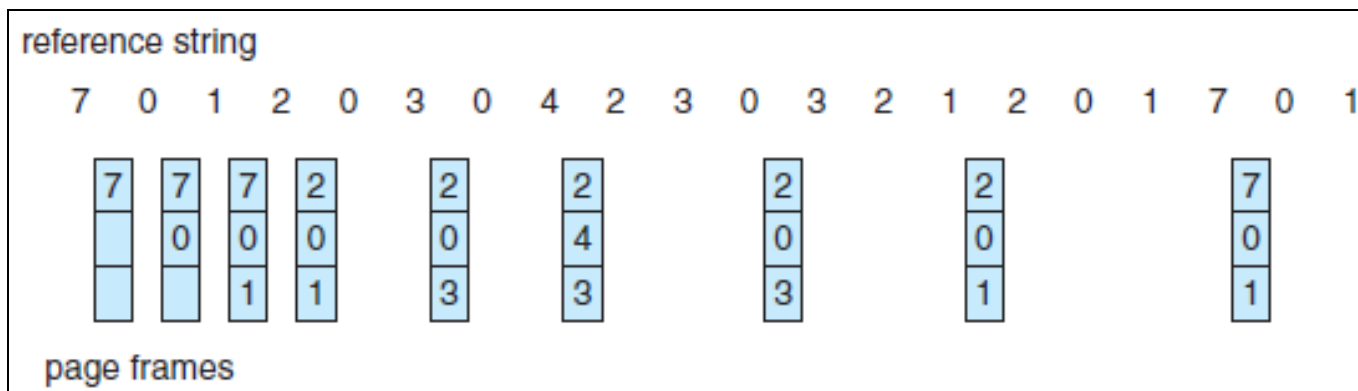
1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

string dizesini ele alırsak.
- 3 çerçeve için 9 sayfa hatası, 4 çerçeve için 10 sayfa hatası olması; çerçeve sayısı artmasına rağmen hata sayısı artmıştır!
- Buna en beklenmedik sonuç, **Belady anormalliği** denir.



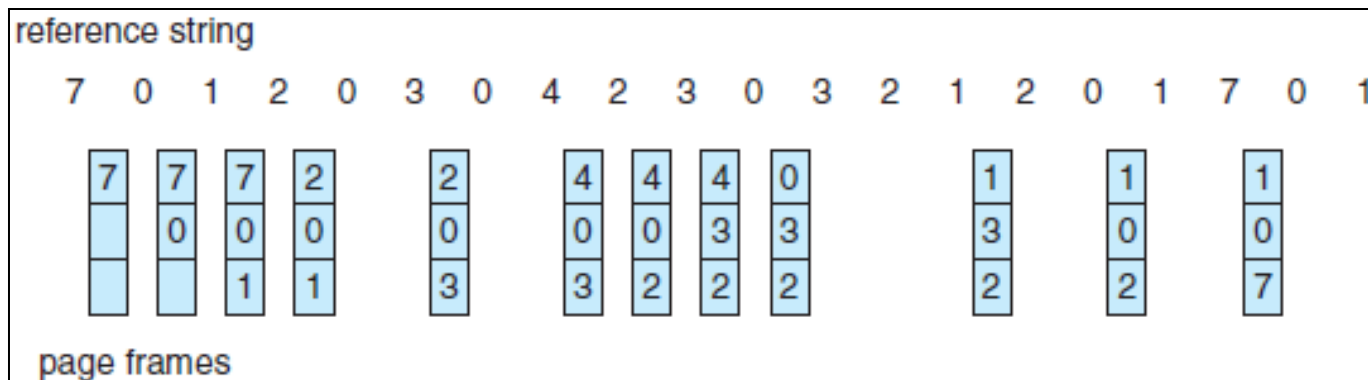
# Page Replacement – Optimal

- Tüm algoritmalar arasında en düşük sayfa hatası oranına sahip ve Belady anormalliğinin asla olmadığı bir algoritma ideal olanıdır.
  - Teori: «En uzun süre kullanılmayacak olan sayfayı değiştir» şeklindedir.
    - Bu fikir sabit sayıda çerçeve için en düşük sayfa hata oranını garanti eder.
- Bu teoriye göre örnek referans dizesi işletilirse;
  - İlk üç referans, üç boş çerçeve için 3 sayfa hatasına neden olur.
  - Sonra P2 referansı P7 ile değiştirilir. Çünkü P7 tekrar 18. zamanda referans edilecek
  - Sonra P3 referansı P1 ile değiştirilir... şeklinde algoritma devam eder.
  - Optimum Sayfa Değiştirme algoritması 9 sayfa hatası ile tamamlanır.



# Page Replacement – LRU

- Optimal algoritması uygulanabilir olmadığından;
  - LRU algoritması; yakın geçmiş, yakın geleceği tahmin etmek için kullanılırsa, en uzun süredir kullanılmayan sayfa değiştirilebilir fikrine dayanmaktadır.
  - Dolayısıyla LRU, her sayfayla o sayfanın son kullanım zamanını ilişkilendirir.
  - Örnek referans dizesi için LRU 12 sayfa hatası üretir.
- İlk 5 hata, optimum değiştirme için oluşan hatalarla aynıdır.
- P4 referans edildiğinde, bellekteki üç çerçeveden P2 en uzun süredir kullanılmadığı için P2 ile değiştirilir. Daha sonra P2 için sayfa hatası tekrar oluşur.
- Bu sorunlara rağmen LRU 12 hatayla, 15 hatalı FIFO'dan daha iyidir.





# Page Replacement – LRU

- LRU iyi bir sayfa değiştirme algoritması olarak kullanılır ancak LRU'nun nasıl uygulanacağı bir sorundur.
  - Çünkü kullanım zamanına göre çerçeveler belirlenmelidir. (İki yöntem kullanılabilir)
- LRU algoritması önemli bir donanım desteğine ihtiyaç duyar.
  - **Counters:** Her page-table kaydına bir «time-of-use» alanı eklenir.
    - CPU'da bir «clock-register» tutulur. Bu register her bellek referansında artırılır.
    - Bir sayfa referanslandığında, «clock-register» o sayfanın «time-of-use» alanına kopyalanır.
    - Böylece, her sayfanın son referans zamanı bilinir. Bu yöntem, uygun sayfayı bulmak için page-table'ın taranmasını ve her bellek erişiminde belleğe yazmayı gerektirir.
  - **Stack:** Bir sayfa referanslandığında, stack'ten çıkarılır ve stack'in en üstüne konur.
    - Böylelikle en son kullanılan sayfa her zaman yığının en üstünde, en az kullanılan sayfa ise her zaman en altta olur.
    - Referans edilen sayfa stack'in ortasında ise, double-linked-list kullanarak ilgili sayfa'nın stack konumu güncellenebilir. Maliyetlidir, ancak page-table araması yoktur.

# Page Replacement – LRU

- Optimum sayfa değiştirme algoritması gibi, LRU'da Belady anormalliğinden etkilenmez
- Optimal gibi RLU algoritması da Stack algoritması adı verilen bir sayfa değiştirme algoritması sınıfına aittir.
  - Bir Stack Algoritması,  $n$  çerçeve için bellekteki sayfa kümesinin her zaman  $n + 1$  çerçeveli bellekte olacak sayfa kümesinin bir alt kümesi olduğunu gösterir.
  - LRU için, bellekteki sayfa grubu en son referans edilen  $n$  sayfa olacaktır.
    - Çerçeve sayısı artırılırsa, bu  $n$  sayfa yine de en son referans edilen sayfalar olacaktır.
- LRU ilave donanım desteği olmadan uygulanamaz.
  - Clock register'ları yada stack güncellemesi, her bellek referansı için gereklidir.
  - Yazılımın bu veri yapılarını güncellemesi için her referans bir interrupt oluşturacaktır.
    - Bu durum her bellek referansını on kat yavaşlatır, dolayısıyla her süreci on kat yavaşlatır.
    - Bu düzeydeki bir ek yükü hiçbir sistem tolere edemez.

# Page Replacement – LRU Temelli Yaklaşımlar

- Pek çok bilgisayar sistemi, orijinal LRU için yeterli donanım desteği sağlamaz.
  - Hiçbir donanım desteği sağlamayan sistemler, FIFO vb. sayfa değişim algoritmalarını kullanmak zorundadırlar.
- Ancak birçok sistem, referans bit (**reference bit**) adı verilen bir donanım desteği sunar.
- Bir sayfanın referans biti, o sayfa her referans edildiğinde donanım tarafından ayarlanır (sayfadaki herhangi bir byte'ın okunması yada yazılması).
  - Referans bitleri, page-table'daki her kayıt (o sürece ait sayfalarla) ile ilişkilendirilir.
- Başlangıçta, tüm bitler işletim sistemi tarafından resetlenir (0).
- Bir süreç yürütülürken, referans edilen sayfayla ilişkili bit, donanım tarafından kurulur (1).
  - Böylece sayfaların kullanım sırası bilinmese de referans bitleri ile hangi sayfaların kullanıldığı belirlenebilir.
- Bu bilgi, LRU sayfa değiştirme algoritmasını referans alan birçok sayfa değiştirme algoritmasının temelini teşkil etmektedir.

# Page Replacement – LRU Temelli Yaklaşımlar

**Additional-Reference-Bits Algorithm:** Referans bitleri düzenli aralıklarla page-table'daki her sayfa girişi için 8 bitlik bir referans bit alanına kaydedilerek (shift registers) ilgili sayfanın kullanım sırasına dair çıkarım yapılabilir.

- Düzenli aralıklarla (örn. 100 ms'de bir timer-interrupt) işletim sistemi her sayfanın referans bitlerini 1 bit sağa kaydırır.
- En-düşük değerlikli bit atılır.
- Sayfaların referans bitleri, son sekiz periyot için sayfa kullanım geçmişini verir.
  - Örneğin, shift-register 00000000 içeriyorsa, sayfa son 8 periyotda kullanılmamıştır.
  - Her periyotta en az bir kez kullanılan bir sayfanın referans bitlerinin değeri 11111111'dir.
  - Geçmiş kayıt değeri 11000100 olan bir sayfa, 01110111 değerine sahip olandan daha yakın zamanda kullanılmıştır.
  - Yani en düşük değere sahip sayfa LRU sayfasıdır ve değiştirilebilir.
  - Ancak bu değerler tekil olamayacağı için en küçük değere sahip birden fazla sayfa olduğunda seçim yapmak için FIFO yöntemi izlenebilir.

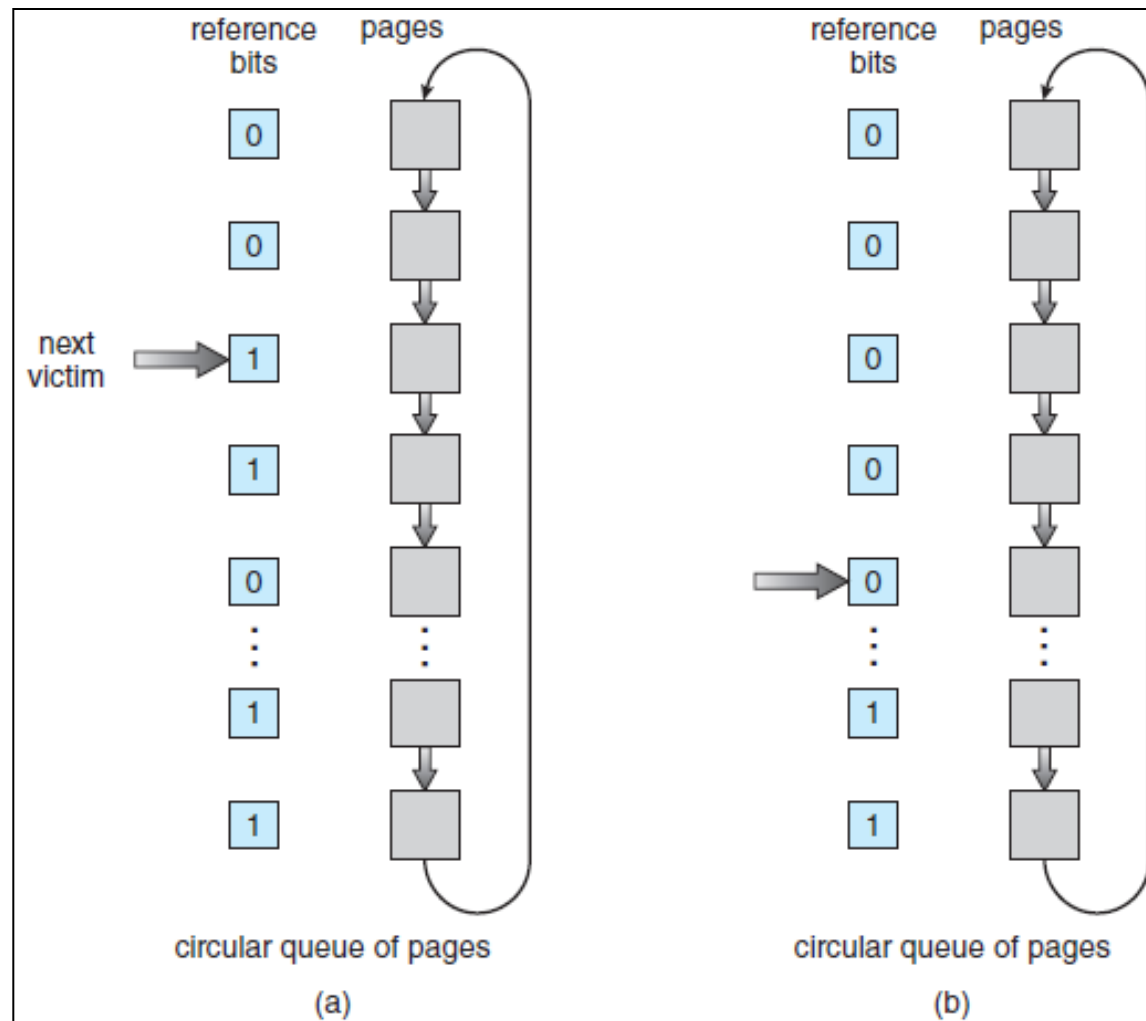
# Page Replacement – LRU Temelli Yaklaşımlar

**Second-Chance Algorithm:** İkinci şans algoritması, bir FIFO algoritmasıdır. Değişim için bir sayfa seçildiğinde, referans biti incelenir.

- Referans biti 0 ise, bu sayfa değiştirilir; 1 ise, sayfaya ikinci bir şans verilir ve FIFO kuyruğundaki bir sonraki sayfa seçilir.
  - Bir sayfa ikinci şansı yakaladığında, referans biti sıfırlanır ancak sayfanın varış saati aktif saat olarak ayarlanır (en son gelen olur).
  - Bu nedenle, ikinci şans verilen sayfa, tüm sayfalar değiştirilene kadar değiştirilmeyecektir.
- İkinci şans algoritması(**clock algoritması** olarak bilinir) döngüsel bir kuyruktur
  - Bir pointer, bir sonraki adımda hangi sayfanın değiştirileceğini gösterir.
  - Bir çerçeveye gerektiğinde, pointer 0 değerine sahip bir sayfa bulana kadar ilerler.
  - Uygun bir sayfa bulduğunda, sayfa değiştirilir ve yeni sayfa dairesel kuyruğa bu pozisyondan eklenir.
  - En kötü durumda (tüm sayfaların referans bitleri 1'ise), pointer tüm kuyruk boyunca dönecek ve her sayfaya ikinci bir şans verecektir.
  - Tüm bitler 1 ise tüm sayfaların bitleri sıfırlanır ve FIFO gibi çalışır.

# Page Replacement – LRU Temelli Yaklaşımlar

## Second-Chance Algorithm:



# Page Replacement – LRU Temelli Yaklaşımlar

**Enhanced Second-Chance Algorithm:** İkinci şans algoritması, referans ve modifiye bitini sıralama için kullanır ve dört (2 bit) olası sınıf tanımlar:

1. (0, 0) Yakın zamanda kullanılmamış ve değiştirilmemiş (en ideal sayfa)
  2. (0, 1) Yakın zamanda kullanılmamış ancak değiştirilmiş (pek iyi değil, çünkü sayfa değiştirilmeden önce ikincil depolama birimine saklanması gerekir)
  3. (1, 0) Yakın zamanda kullanılmış ancak değiştirilmemiş (muhtemelen kısa süre sonra tekrar kullanılacak)
  4. (1, 1) Yakın zamanda kullanılmış ve değiştirilmiş – (muhtemelen yakında tekrar kullanılacak ve değiştirilmeden önce ikincil depoya saklanması gerekir)
- Sayfa değişimi istendiğinde, clock algoritmasındaki yapı kullanılır.
    - Ancak seçilen sayfanın referans bitinin 1 olup/olmadığına bakmak yerine, o sayfanın ait olduğu sınıfa bakılır.
    - Sınıf değeri en düşük sayfalar değiştirilir.
    - Ancak döngüsel olarak kuyruğun birkaç kez taranması gerekebilir.
    - Bu algoritma ile clock algoritması arasındaki en büyük fark, I/O sayısını azaltmak için değiştirilmiş sayfalar yerine önceliğin değiştirilmemiş sayfalara verilmesidir.

# Page Replacement – Counting-Based Page Rep.

- Sayfa değişimi için kullanılabilecek başka bir algorithma ise her sayfaya yapılan referans, bir sayaçta (counter) tutulur ve bu sayaç değerine göre;
- **En az kullanılan (least frequently used-LFU):** En düşük sayaç değerine sahip sayfanın değiştirilmesini önerir.
  - **Aktif olarak kullanılan bir sayfa daha fazla referans edilir fikrine dayanır.**
    - Ancak, bir sürecin ilk aşamasında bir sayfa yoğun bir şekilde kullanıldığında, daha sonra bir daha asla kullanılmadığında sorunlar ortaya çıkar.
    - Yoğun bir şekilde kullanıldığı için sayacı fazladır ancak artık ihtiyaç duyulmasa da hafızada kalır.
    - Bu soruna karşı üretilen çözümlerden biri, sayaç'ın düzenli aralıklarla 1 bit sağa kaydırarak değerinin azaltılmasıdır.
- **En sık kullanılan (most frequently used-MFU)** En düşük sayaç değerine sahip sayfanın muhtemelen belleğe yeni getirildiği ve henüz kullanılmadığı fikrine dayanmaktadır.
- Bu algoritmaların uygulanması maliyetlidir ve Optimum sayfa değişimine yaklaşılamazlar.



# Page Replacement – Page Buffering Algorithms

- Sistemler genellikle sabit bir boş çerçeve havuzu (free-frames pool) tutar.
  - Bir sayfa hatası oluştuğunda, havuzdan boş çerçeve seçilir.
  - Referans edilen sayfa, değişim için seçilen sayfa diske yazılmadan önce boş çerçeveye yüklenir. Böylece sürecin en kısa sürede tekrar yürütülmesi sağlanır.
  - Değişim için seçilen sayfa daha sonra ikincil depoya yazılarak boşaltılan çerçeve, havuza iade edilir.
- Bu fikrin bir varyantında, modifiye edilmiş sayfaların listesi tutulur.
  - Paging device boşta kaldığında, modifiye sayfalar ikincil depoya yazılır ve modifiye biti sıfırlanır.
    - Böylece, değiştirilmek üzere bir sayfa seçildiğinde diske yazılması gerekmez.
- Başka bir varyantta ise, her çerçevede hangi sayfanın tutulduğunun bilindiği bir boş çerçeve havuzu tutarak;
  - Eski sayfa, tekrar referans edilirse serbest çerçeve havuzundan doğrudan tekrar kullanılmak üzere çağrılabilir. Bu durumda I / O gerekmez.
- UNIX'in bazı sürümleri bu yöntemi ikinci şans algoritması ile birlikte kullanır.

# Page Replacement – Uygulamalar Özelinde

- Bazı durumlarda; verilere işletim sisteminin sanal belleği üzerinden erişen uygulamalar, işletim sisteminin arabelleğini kullanmadığı durumlara göre daha kötü performans verirler.
  - Örneğin, kendi arabellek ve cache yönetimini yapan bir veritabanı sunucusu bellek yönetimini, genel amaçlı kullanım için geliştirilen algoritmaları uygulayan işletim sisteminden daha iyi yapar.
  - Çoğunlukla büyük sıralı disk okuması/yazması gerçekleştiren veri ambarlarında; eski sayfaları kaldırıp yeni sayfaları koruyan salt LRU algoritması verimsiz olabilir.
    - Çünkü uygulama eski sayfaları yeni sayfalara göre daha önce okuyacaktır (sıralı okumalarda).
    - Burada, MFU aslında LRU'dan daha verimli olacaktır.
  - Bu tür sorunlar nedeniyle işletim sistemleri bazı yazılımlara, herhangi bir dosya sistemi veri yapısı olmaksızın, ikincil bir depolama bölümünü kullanım izni verir.
  - Bu **raw disk** olarak adlandırılır ve bu diziye yapılan I/O, **raw I/O**'dur.
    - Raw I/O, tüm dosya sistemi servislerini atlar.

# Allocation of Frames – Çerçeve Tahsisi

- Sınırlı belleğin (free) süreçler arasında nasıl tahsis edileceği önemlidir.
  - 93 boş çerçeve varsa bu çerçeveler iki süreç arasında nasıl tahsis edilir?
- 128 çerçeveye sahip bir sistem için;
  - İşletim sistemi, kullanıcı süreçlerine 93 çerçeve, OS sistemine 35 çerçeve tahsis etsin,
    - Yalın Sayfalama İsteği altında 93 çerçevenin tümü başlangıçta free-frame list'e alınır.
    - Bir kullanıcı süreci yürütülmeye başladığında, bir dizi sayfa hatası üretilecektir.
    - İlk 93 sayfalık hataların tümü, serbest çerçeve listesinden karşılanacaktır.
    - Serbest çerçeveler tükendiğinde, bellekteki sayfadan biri 94'üncü sayfa ile değiştirilecektir.
      - Hangisinin değiştirileceğini seçmek için bir sayfa değiştirme algoritması kullanılacaktır.
- Çerçeve tahsisi için sistemin tüm buffer'ını, free-frame list'e alması istenebilir.
- Yada free-frame list'te her zaman 3 boş çerçeve tutulması istenebilir.
  - Çerçeve tahsisi ve free-frame list için başkaca yöntemler de uygulanabilir.
- Ancak temel strateji: Süreçlere boş bir çerçeve tahsis etmektir.

# Minimum Çerçeve Sayısı

- Çerçeve tahsisindeki en önemli parametre mevcut çerçeve sayısından daha fazla çerçeve tahsis edilememesidir (page sharing hariç).
- Ancak en azından minimum sayıda çerçeve ayrılmalıdır.
  - Bu stratejide, minimum sayıda çerçeve tahsisi performans içindir.
  - Ancak bir sürece tahsis edilen çerçeve sayısı azalıkça page-fault artar, süreç yavaşlar.
    - Herhangi bir komutun referans edebileceği sayfaları tutacak yeterli sayıda çerçeve olmalıdır.
    - Örneğin, dolaylı adreslemeye izin veren bir sistemde süreç başına 3 çerçeve gerekir.
      - Frame-16'daki bir load komutu, frame-23'ü referans eden frame-0'daki bir adresi refere edebilir.
    - Minimum çerçeve sayısını o sistemin mimarisi belirler,
      - Örn. move komutu birden fazla operant içeriyorsa bazı adresleme modları için 2 çerçeveyi aşabilir.
        - Operantların her biri dolaylı referanslar içeriyorsa 6 çerçeve ihtiyacı doğar.
    - Intel 32-64 bit mimarileri move komutu için transferin yalnızca register-register, register-bellek arasında yapılabilmesine izin verir. Doğrudan bellek-bellek taşınmasına izin vermez.
    - Dolayısıyla bir süreç için gereken minimum çerçeve sayısını sınırlar.

**Süreç başına minimum çerçeve sayısını mimari belirlerken, maksimumu fiziksel bellek belirler.**

# Allocation Algorithms – Tahsis Algoritmaları

- m adet çerçeveyi n süreç'e tahsis etmenin en kolay yolu her sürece eşit oranda ( $m/n$ ) paylaşmaktır. (İşletim sisteminin kendine ayırdıkları hariç)
  - Örn, 93 çerçeve 5 sürece, 18'er tane tahsis edilir ve 3 çerçeve free-frame list'e kalır.
  - Bu yöntemle eşit tahsis (**equal allocation**) denir.
- Alternatif olarak süreçlerin farklı oranda belleğe ihtiyaç duyacağı kabul edilir.
  - Örn, 1 KB çerçeve boyutuna sahip bir sistemde 2 süreç yürütülecek olsun;
    - 10 kb'lık bir süreç ve 127 kb'lık bir süreç için 62 boş çerçeve sunan bir sistem;
      - Her sürece 31 çerçeve tahsis etseydi mantıklı olmazdı. 1. süreç 10 çerçeve kullanır 21'i boşa giderdi.
  - Bu nedenle her süreç için boyutuna göre kullanılabilir belleğin tahsis edildiği orantılı tahsis (**proportional allocation**) kullanılabilir.
    - $p_i$  süreci için gereken sanal bellek boyutu  $s_i$  olursa;  $S = \sum s_i$
    - Toplam çerçeve sayısı m ise,  $a_i$  kadar çerçeve  $p_i$ 'ye tahsis edilir;  $a_i = (s_i / S) \times m$
    - Burada her  $a_i$  toplamı m'yi geçmeyen ancak komut setinin gerektirdiği minimum çerçeve sayısından büyük bir tam sayı olacak şekilde ayarlanmalı.

# Allocation Algorithms – Tahsis Algoritmaları

- Örn, orantılı tahsis ile, 62 çerçeveyi, birincisi 10 sayfadan diğeri 127 sayfadan oluşan iki süreç arasında tahsis etmek isteseydik;
  - $10 / 137 \times 62 \approx 4$  ve
  - $127 / 137 \times 62 \approx 57$ .
  - Böylece, her sürece çerçeveler eşit değil, “ihtiyacı” kadar tahsis edilmiş oldu.
- Hem eşit hem de orantılı tahsiste, çerçeve tahsisi, multiprogramming seviyesine göre değişebilir.
  - Çoklu programlama seviyesi artarsa, her süreç yeni süreç için gereken belleği sağlamak üzere bazı çerçeveleri bırakacaktır. Terside mümkün!
- Eşit yada orantılı tahsis algoritmalarında, yüksek öncelikli bir süreç düşük öncelikli bir süreç gibi değerlendirilmiştir.
  - Ancak, yüksek öncelikli sürecin yürütümünü hızlandırmak için daha fazla bellek vermek istendiğinde;
    - Orantılı tahsis algoritmasında, mevcut çerçeveleri süreçlerin göreceli boyutlarına oranlamak yerine, süreçlerin öncelikleri ve boyutlarının kombinasyonuna oranlayabiliriz.

# Global ve Yerel Tahsis

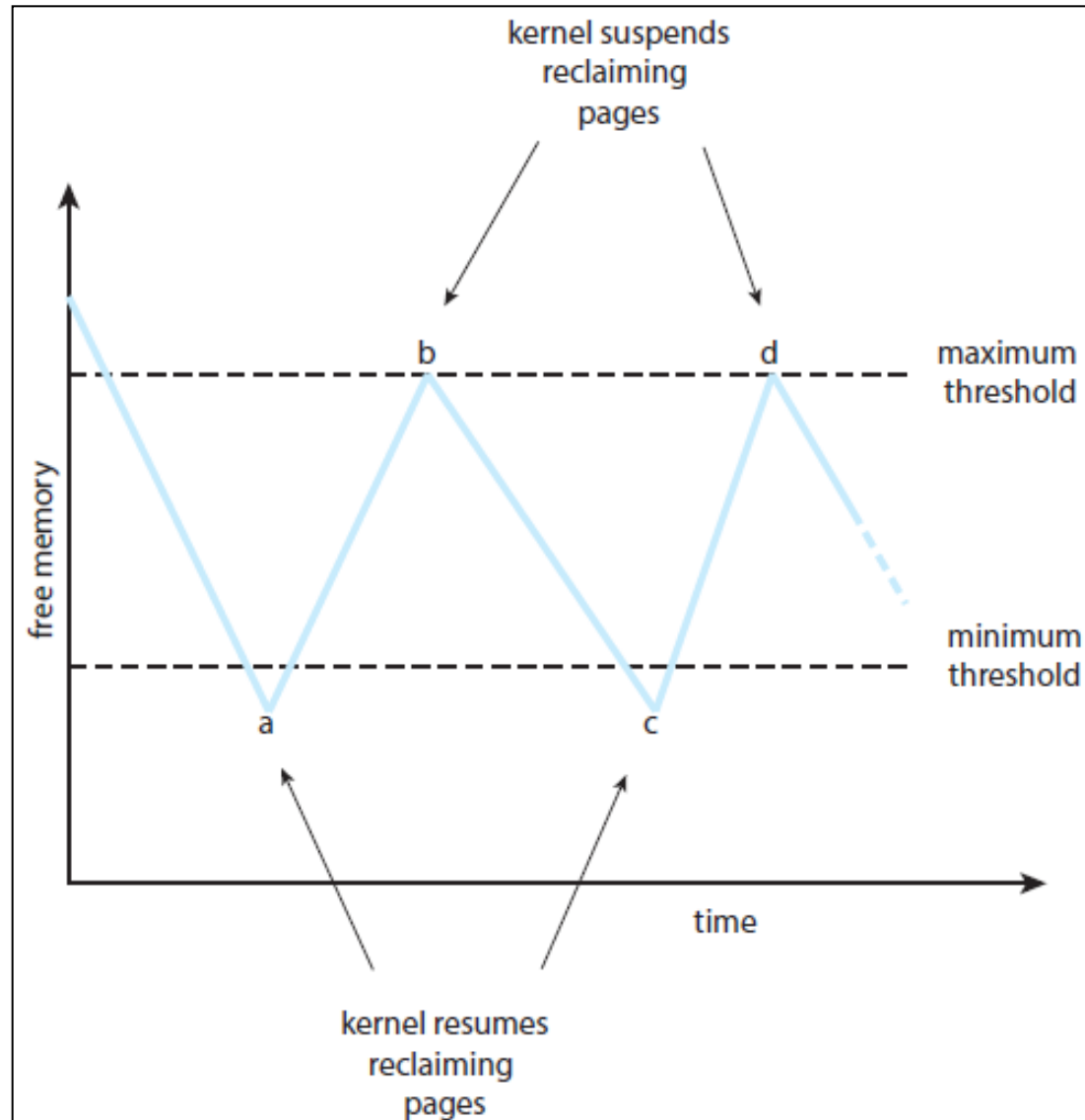
- Çerçevelerin çeşitli süreçlere tahsisinde önemli bir faktör, sayfa değişimidir.
  - Sayfa değiştirme algoritmaları iki kategoride ele alınabilir: Global değiştirme (**global replacement**) ve yerel değiştirme (**local replacement**).
- **Global değiştirme**, bir sürecin mevcut çerçeve kümesinden bir çerçeve seçebilmesini ifade eder.
  - Bu çerçeve o anda başka bir sürece tahsis edilmiş olsa bile seçilebilir (bir süreç diğer süreçten çerçeve alabilir).
- **Yerel değiştirme**, her sürecin yalnızca kendine ayrılan çerçeve kümesinden seçim yapmasını gerektirir.
  - Örneğin, yüksek öncelikli süreçleri düşük öncelikli süreçlerden çerçeve seçmesine izin veren bir tahsis tekniği kullanıldığında;
    - Bir süreç kendi çerçeveleri yada düşük öncelikli süreçlere ait çerçevelerde seçim yapabilir.
  - Yerel değiştirme algoritmalarında bir sürece tahsis edilen çerçeve sayısı değişmezken global değiştirme algoritmalarında bir sürece tahsis edilen çerçeve sayısı artabilir.
    - Diğer süreçler ondan çerçeve almazsa!

# Global ve Yerel Tahsis

- Global değiştirme de önemli bir nokta, bir sürecin bellekteki sayfa kümesi, o sürecin davranışına değil, diğer süreçlerin davranışına bağlı olmasıdır.
  - Bu nedenle süreç, harici koşullar nedeniyle oldukça farklı şekilde davranabilir,
    - Örneğin, bir yürütüm için 0,5 saniye, bir sonraki yürütüm için 4,3 saniye gibi!
- Yerel değiştirme algoritmasında böyle bir durum oluşmaz.
  - Ancak yerel değiştirme, az kullanılan sayfaları serbest bırakmayarak bir sürecin çalışmasını engelleyebilir.
  - Bu nedenle global değiştirme, daha yüksek verim sunduğu için yaygın kullanılır.
- Global bir sayfa değiştirme tekniği free-frame list'den bellek isteklerini karşılar.
  - Ancak free-frame list bitmeden önce (bir eşik değeri), sayfa değiştirme tetiklenir.
    - Böylece yeni istekleri karşılayabilmek için her zaman yeterli boş çerçevenin olmasını sağlamaya çalışır.
    - Free-frame list eşik değerin altına düştüğünde, sistemdeki tüm süreçlerden sayfaları geri çağıran (çekirdek hariç) bir çekirdek yordamı (**Reapers**) tetiklenir.
    - Boş bellek miktarı maksimum eşiğe ulaştığında tetiklenen Reapers askıya alınır.



# Global ve Yerel Tahsis



# Global ve Yerel Tahsis

- Çekirdek rutini (reapers) sayfa geri talep algoritması (Page Replacement) olarak herhangi bir yöntemi tercih edebilir. (Genelde bir tür LRU varyantı kullanılır)
  - Ancak reapers rutini serbest çerçeve sayısını istenilen eşik değerinin üstünde tutamadığı durumda;
    - Daha agresif davranması gerekebilir ve örneğin ikinci şans (second-chance) algoritmasını durdurup saf bir FIFO kullanabilir.
  - Örneğin Linux, boş bellek miktarı çok düşük seviyelere indiğinde, **out-of-memory (OOM) killer** olarak bilinen rutin ile bir süreci sonlandırarak belleği serbest bırakır.
    - Sonlandırılacak süreci seçmek için Linux, süreçlerin sahip olduğu OOM puanına bakar.
      - OOM puanı yüksek olan sürecin OOM Killer tarafından seçilme olasılığı artar.
    - OOM puanları bir sürecin kullandığı bellek yüzdesine göre hesaplanır.
    - Yüzde yüksekse OOM puanı da yükselir.
    - Örneğin pid değeri 2500 olan bir sürecin oom puanı «cat /proc/2500/oom\_score» şeklinde görüntülenebilir.
  - Ayrıca boş hafıza (minumum/maksimum) eşik değerleri değiştirilebilir. Ancak genellikle sistemdeki fiziksel bellek miktarına göre yapılandırılırlar.

# Oku – Büyük/Küçük Sayfa Hataları

Referans edilen bir sayfa, sürecin adres uzayında olmadığına sayfa hatası oluşur. İşletim sistemleri sayfa hatalarını iki başlık altında toplar: büyük ve küçük hatalar. (Windows, büyük ve küçük hataları hardware ve soft hatalar olarak ifade eder.) Büyük sayfa hatasının servisi, istenen sayfanın ikincil depolama biriminden boş bir çerçeveye okunmasını ve page-table'ın güncellenmesini gerektirir. Talep sayfalamada başlangıçta yüksek oranda büyük sayfa hataları oluşur.

Küçük sayfa hataları, sürecin referans ettiği sayfanın fiziksel bellekte olduğu ancak sürecin adres uzayında eşlemesinin olmadığı durumlarda ortaya çıkar. Bu durum iki şekilde meydana gelebilir. İlk olarak, bir süreç bellekte bulunan paylaşılan bir kütüphaneye başvurabilir. Ancak sürecin page-table'ında eşlemesi yoktur. Bu durumda, page-table bellekteki mevcut sayfayı referans etmek için güncellenir. İkinci nedeni ise, bir sayfa bir süreçten geri talep edilmiş ve free-frame list'e yerleştirilmiştir. Ancak sayfa henüz sıfırlanıp başka bir sürece atanmadığından bu çerçeve free-frame list'den alınır ve sürece tekrar atanır. Görülebileceği gibi, küçük sayfa hatasını çözmek, genellikle büyük sayfa hatasını çözmekten çok daha az zaman alır.

Bir Linux sisteminde büyük ve küçük sayfa hatalarının sayısı, "ps -eo" komutunu kullanarak görülebilir. Bu komutun örnek çıktısı aşağıdaki gibidir:

| MINFL  | MAJFL | CMD                             |
|--------|-------|---------------------------------|
| 186509 | 32    | /usr/lib/systemd/systemd-logind |
| 76822  | 9     | /usr/sbin/sshd -D               |
| 1937   | 0     | vim 10.tex                      |
| 699    | 14    | /sbin/denetimd -n               |

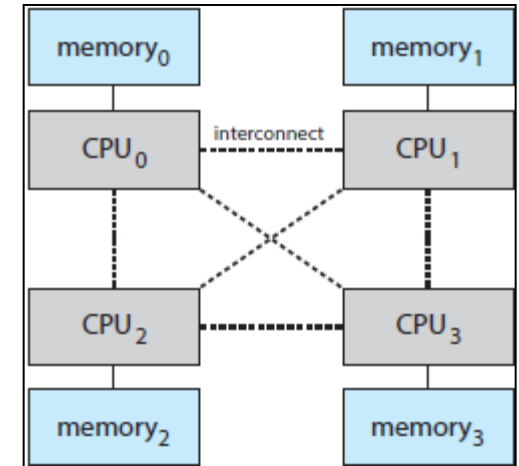
Burada büyük sayfa hatalarının sayısı düşükken, küçük sayfa hatalarının sayısının fazla olması önemlidir. Bu, Linux süreçlerinin muhtemelen paylaşılan kütüphaneleri önemli ölçüde kullandığını gösterir. Çünkü bir kütüphane belleğe yüklendikten sonra, sonraki sayfa hataları küçük hatalardır.

# Non-Uniform Bellek Erişimi - NUMA

- Şu ana kadar sanal bellek içeriğinin oluşturulduğu ana belleğin eşit verimlilikle erişilebildiğini varsaydık.
  - NUMA sistemlerde her biri kendi yerel belleğine sahip birden çok CPU bulunur.
    - CPU'lar, ortak bir sistem ara bağlantısı üzerinden diğer CPU yerel belleklerine erişebilir.
    - NUMA sistemler, ana belleğe tüm erişimlerin eşit olduğu diğer sistemlerden daha yavaştır.
      - Ancak NUMA sistemler daha fazla CPU barındırabilir ve daha yüksek verim ve paralellik sağlar.
  - Bu nedenle hangi sayfa çerçevelerinin hangi konumlarda depolandığı NUMA sistemlerin performansını önemli ölçüde etkiler.
    - Bellek tahsis algoritmasının NUMA'yı hesaba katmadığı durumda CPU bellek erişimleri uzar
      - Dolayısıyla bir süreç sayfa hatası aldığında NUMA'ya duyarlı bir sanal bellek sistemi sürecin çalıştığı CPU'ya olabildiğince yakın bir çerçeve tahsis edecektir.
      - NUMA'yı hesaba katmak için Dispatcher/Scheduler her sürecin çalıştığı CPU'yu bilmelidir. (**Affinity**)
        - Bir sonraki planlamada olabildiğince bir önceki yürütüldüğü CPU'ya planlanmaya çalışılır (**soft affinity**)
        - Hard affinity'de olabilir.
    - Olaya thread'ler de girdiğinde durum çok daha karışık bir hal alacaktır.
      - Örneğin çok sayıda thread içeren bir süreç bir çok farklı CPU'da planlanmış olabilir.

# Non-Uniform Bellek Erişimi - NUMA

- Linux öncelikle planlama etki alanlarının (scheduling domains) hiyerarşisini tanımlar.
  - Linux CFS planlayıcısı, thread'lerin farklı etki alanları arasında geçiş yapmasına izin vermez (bellek erişim cezalarına neden olur).
- Linux her NUMA için ayrı bir free-frame list tutar.
  - Böylece bir thread'in çalıştığı düğümden bellek tahsisi sağlar.
- Solaris, çekirdekte **lggroups** (locality groups-yerel gruplar) oluşturur.
  - Her **lggroup**, CPU'ları ve bellekleri yerel grup olarak tanımlar.
    - Gruptaki her CPU, tanımlanmış bir gecikme aralığı içinde gruptaki bir belleğe erişebilir.
    - Linux'te scheduling domains hiyerarşisine benzer, gruplar arasındaki gecikme miktarına dayalı bir **lggroup** hiyerarşi vardır.
  - Solaris, bir sürecin tüm iş parçacıklarını ve tüm belleğini bir **lggroup** içinde planlamaya/tahsis etmeye çalışır.
  - Bu olmazsa, ihtiyaç duyulan kaynakların geri kalanı için yakındaki lggroups'ları seçer.



# Thrashing - Çökme

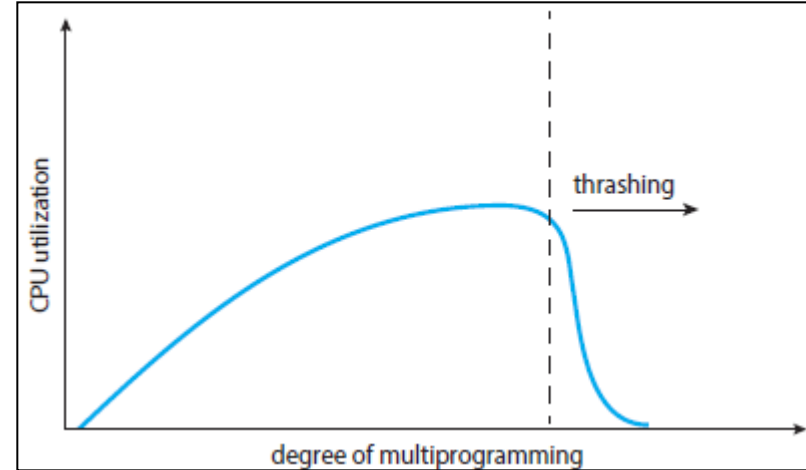
- Bir süreç, çalışma kümesindeki (working set) sayfaları tutmak için ihtiyaç duyduğu minimum çerçeve sayısına sahip değilse;
  - Yani sürecin yeterli çerçevesi yoksa,
  - Süreç hızlı bir şekilde sayfa hatası verecektir.
  - Bu noktada aslında aktif olarak kullanılan bir çerçeve sayfa değişimi ile serbest bırakılır.
  - Ancak, tüm çerçeveler aktif olarak kullanıldığı için, serbest bırakılan sayfaya tekrar ihtiyaç duyulacak ve bir önceki adımdaki gibi başka bir aktif çerçeveyle sayfa değişimine gidilecektir.
  - Sonuç olarak, süreç hemen/tekrar geri getirmesi gereken sayfalar nedeniyle tekrar tekrar sayfa hatası verecektir.
- Bu yüksek sayfalama etkinliğine **Thrashing (Çökme)** denir.
  - Bir süreç, sayfalama aktivitesine, yürütüm aktivitesinden daha fazla zaman harcıyorsa, çökme oluşuyor demektir.
  - Tahmin edilebileceği gibi, çökme ciddi performans sorunlarına neden olur.

# Cause of Thrashing - Çökme Nedeni

- İşletim sistemleri CPU utilization'ını izler ve utilization düşükse, sistem multiprogramming düzeyini artırmak için yeni bir süreç yürütümünü başlatır.
- Bir **global sayfa değiştirme (page-replacement) algoritması** kullanıldığında, sayfaların hangi sürece ait olduğuna bakılmaksızın sayfalar değiştirilebilir.
  - Bir süreç yürütülürken daha fazla çerçeveye ihtiyaç duyarsa sayfa hatası verir ve diğer süreçlerden çerçeve almaya başlar.
  - Ancak bu süreçler de bu sayfalara ihtiyaç duyarlar ve sayfa hatasına neden olarak diğer süreçlerden çerçeveler alırlar.
    - Sayfa hatası veren süreçler takaslamalar için Paging Device'ı (ikincil depo) kullanırlar.
    - Bu süreçler Paging Device kuyruğuna girdiklerinden, hazır iş kuyruğu (ready queue) boşalır.
    - Süreçler Paging Device kuyruğunu beklerken CPU utilization'ı azalır. (I/O waiting)
    - CPU Scheduler, azalan utilization'ı görür ve multiprogramming düzeyini artırır.
    - Yeni süreç, diğer süreçlerden çerçeveler alarak başlamaya çalışır ama bu daha çok sayfa hatasına ve daha çok Paging Device kuyruğuna neden olur.
    - Sonuçta CPU Utilization düşer, Scheduler yeni süreçler yürütür ve bu sarmal devam eder.
    - Sayfa hata oranı aşırı artar. Etkin bellek erişim süresi düşer.

# Cause of Thrashing - Çökme Nedeni

- Multiprogramming derecesi arttıkça, CPU kullanımı da maksimuma ulaşana kadar artar.
- Multiprogramming derecesi daha da artırılırsa, thrashing meydana gelir.
- CPU kullanımı keskin bir şekilde düşer.
- Thrashing'i durdurmak için Multiprogramming derecesi düşürülmelidir.
- **Bir yerel (local replacement) sayfa değiştirme algoritması (veya priority replacement algoritması) kullanıldığında;**
  - Her süreç yalnızca kendine tahsis edilmiş çerçeve kümesinden seçim yapar.
    - Bu nedenle bir süreç thrashing'e neden olursa diğer süreçlerden çerçeve alamaz ve diğer sürecin çökmesine neden olamaz.
    - Ancak süreçler çökerken, paging device kuyruğuna gireceklerdir.
    - Paging device kuyruğundaki yoğunluk nedeniyle bir sayfa hatası için ortalama servis süresi artacaktır.
    - Böylece etkin erişim süresi thrashing yaşamayan bir süreç için de artacaktır.
  - Dolayısıyla çökmeyi engellemek için bir süreç ihtiyaç duyduğu kadar çerçeve verilebilmelidir.



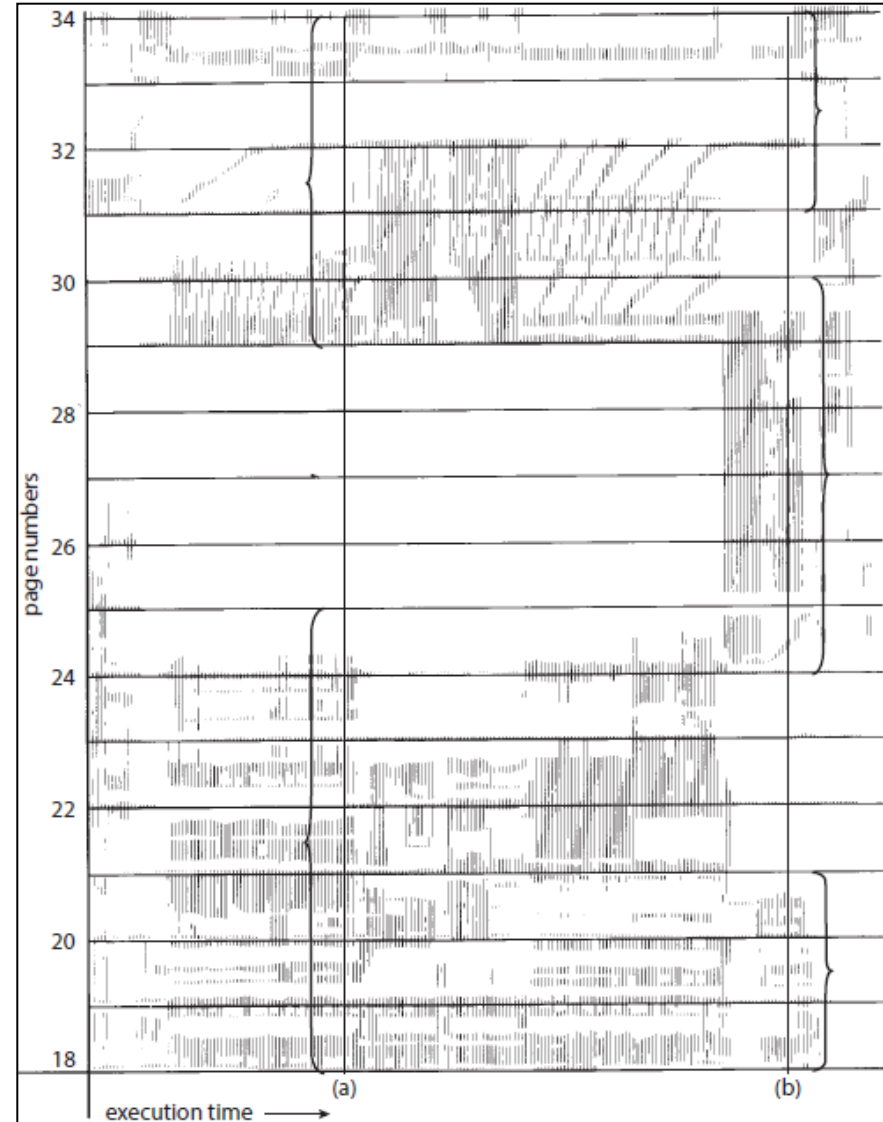


# Cause of Thrashing - Çökme Nedeni

- Bir sürecin ihtiyaç duyduğu (gerçekte kaç frame kullandığı) çerçeve sayısını belirlemenin bir yolu **locality model** (yerellik modeli) olarak ifade edilir.
  - Yerellik modeli, bir süreç yürütüldüğünde yerellikten yerelliğe doğru hareket ettiğini kabul eder.
  - Yerellik (locality), aktif olarak birlikte kullanılan bir sayfa dizidir.
  - Çalışan bir program genellikle örtüşen birkaç farklı lokasyondan oluşur.
  - Örneğin, bir fonksiyon çağrıldığında yeni bir locality'yi tanımlar.
  - Bu locality'de, bu fonksiyonun komutlarına, yerel değişkenlerine ve global değişkenlerin bir alt kümesine referanslar yapılır.
  - Fonksiyondan çıkıldığında, fonksiyonun yerel değişkenleri ve komutları artık aktif olmadığından, süreç bu locality'yi terk eder.
  - Dolayısıyla bir sürece o anki locality'sini barındıracak kadar çerçeve tahsis edilirse;
    - Süreç, o anki yerelliğinin gerektirdiği sayfalar için başlangıçta hata verecektir ancak daha sonra sürecin locality'si değişene kadar sayfa hatası vermeyecektir.

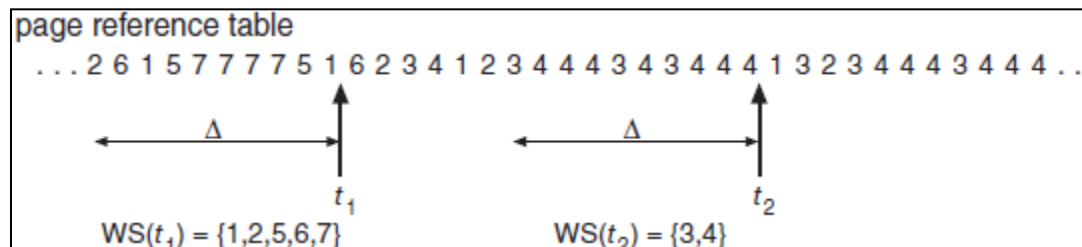
# Cause of Thrashing - Çökme Nedeni

- Bir sürecin locality'sinin zaman içinde nasıl değiştiği;
- (a) anında, yerellik {18, 19, 20, 21, 22, 23, 24, 29, 30, 33} sayfalar kümesidir.
- (b) anında, yerellik {18, 19, 20, 24, 25, 26, 27, 28, 29, 31, 32, 33} olarak değişir.
- Bazı sayfalar (18, 19 ve 20) her iki yerelliğin de parçası olabilir.
- Locality'ler program ve veri yapıları tarafından tanımlanır.
- Locality Model, tüm programların temel bellek referans yapısını ortaya koyar.
- Ayrıca **Locality model**, bir verinin önbelleğe alınmasının arkasındaki temel ilkedir.
  - Aksi durumda rastgele veriler cache'lenmiş olurdu ve işe yaramazdı.



# Working-Set Model – Çalışma Kümesi

- **Working-Set modeli**, locality'ye dayanmaktadır.
  - Bu model, **working-set window**'u tanımlamak için bir  $\Delta$  (delta) parametresi kullanır.
  - $\Delta$  birim içerisindeki sayfa referanslarını inceler. (son 20 referans gibi)
  - En son  $\Delta$  sayfa referanslarındaki sayfalar, **working-set**'dir.
    - Bir sayfa aktif kullanımdaysa, working-set içerisinde olacaktır.
    - Kullanılmıyorsa, yani son referansından sonra  $\Delta$  sayfa referansı varsa workin-set'ten düşer.
  - Bu nedenle working-set, program locality'sinin yaklaşık değeridir.
  - Örn.  $\Delta = 10$  bellek referansı ise,
    - $t_1$  zamanındaki working-set  $\{1, 2, 5, 6, 7\}$  'dir.
    - $t_2$  zamanındaki working-set  $\{3, 4\}$  olacaktır.



# Working-Set Model – Çalışma Kümesi

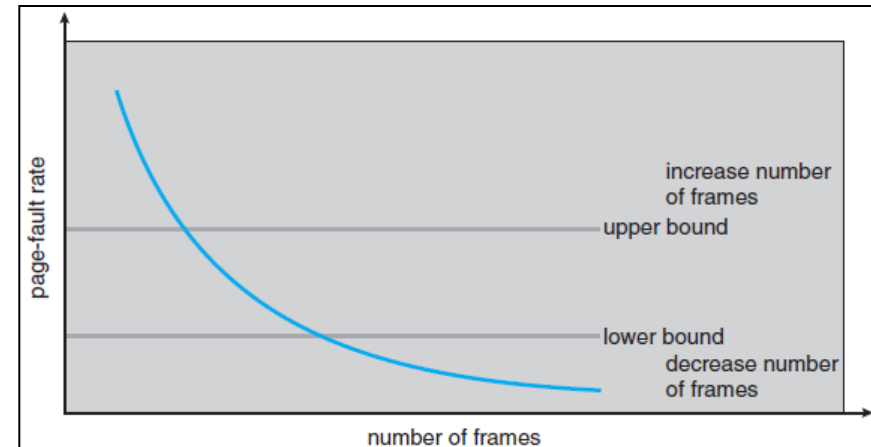
- Working-set'in doğruluğu  $\Delta$  parametresinin seçimine bağlıdır.
  - $\Delta$  çok küçükse, locality'yi yeteri kadar kapsamaz,
  - $\Delta$  çok büyükse, farklı locality'lerle kesişir.
    - $\Delta$  sonsuz ise working-set, sürecin yürütümü boyunca referans ettiği tüm sayfalar kümesidir.
  - Burada önemli olan ne kadar büyük bir working-set seçileceğidir.
    - Sistemdeki her süreç için working-set boyutunu  $WSS_i$  şeklinde ifade edersek;  $D = \sum WSS_i$ 
      - Burada D, talep edilecek toplam çerçeve sayısını ifade etmektedir.
      - Her süreç working-set'teki sayfaları aktif olarak kullandığından  $WSS_i$  kadar çerçeveye ihtiyacı vardır.
      - $D > m$  ise, bazı süreçler ihtiyaç duyduğu çerçeveyi alamayacağı için Trashing olacaktır.
  - $\Delta$  seçildikten sonra işletim sistemi her sürecin working-set'ini izler ve ilgili sürece working-set boyutuna uygun çerçeveyi tahsis eder.
    - Fazladan boş çerçeve varsa da başka süreçlerin çalıştırılmasına izin verilir.
    - Toplam working-set, kullanılabilir toplam çerçeve sayısını aşarsa, bir süreç askıya alınır.
      - Sürecin sayfaları swap'lenir ve boşaltılan çerçeveler diğer süreçlere tahsis edilir.

# Working-Set Model – Çalışma Kümesi

- Working-Set modeli, multiprogramming derecesini olabildiğince yüksek tutarken Trashing'i de önler.
  - Working-set modelindeki temel zorluk, working-set'i takip etmektir.
    - Working-Set Window hareketli bir penceredir.
    - Her sayfa referansında, yeni sayfa referansı kümeye eklenir ve eski referanslar listeden çıkar
- Working-Set modeli zamanlanmış bir kesme ve bir referans-bit ile tahmin edilebilir.
  - Örneğin,  $\Delta$  değerinin 10.000 referans olduğu ve her 5.000 referansta bir kesme oluştuğunu varsayarsak;
    - Her kesmede her sayfa için **referans-bit** değeri belleğe kopyalanır ve temizlenir.
    - Bir sayfa hatası olursa, bir sayfanın son 10.000 ila 15.000 referans içinde kullanılıp kullanılmadığını belirlemek için geçerli referans-biti ve bellekte tutulan önceki referans biti incelenebilir.
    - Kullanılmışsa, bu bitlerden en az biri aktif , kullanılmamışsa, bu bitler pasif olacaktır.
    - En az bir biti aktif olan sayfaların working-set'de olduğu kabul edilebilir.

# Page-Fault Frekansı

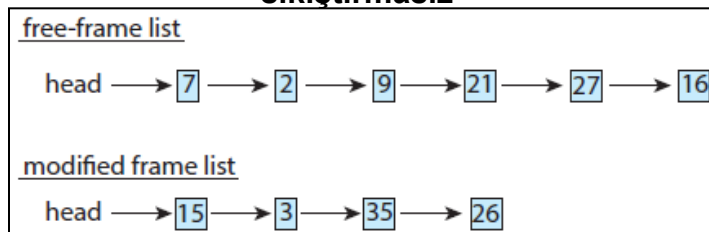
- Working-Set, trashing'i kontrol etmek için çok hantal bir yoldur.
- Onun yerine sayfa hatası sıklığı (**page-fault frequency - PFF**), trashing için doğrudan bir yaklaşım sunar.
  - Thrashing yüksek sayfa hatası oranına neden olduğundan bu oran kontrol edilebilir.
    - Sayfa Hatası Oranı çok yüksekse, sürecin daha fazla çerçeveye ihtiyaç duyduğu anlaşılır.
    - Sayfa Hatası Oranı çok düşükse, sürecin fazladan çerçevesi olduğu anlaşılır.
  - Sayfa hatası oranında üst ve alt sınırlar oluşturulabilir.
    - Sayfa hatası oranı üst sınırı aşarsa, sürece yeni çerçeve tahsis edilir.
    - Sayfa hatası oranı alt sınırın altına düşerse, süreçten çerçeve çıkarılır.
    - Böylece, trashing'i önlemek için sayfa hata oranını doğrudan kullanılabilir.
    - Ancak tahsis edecek çerçeve bulunamazsa süreç tamamen swap-out edilebilir.



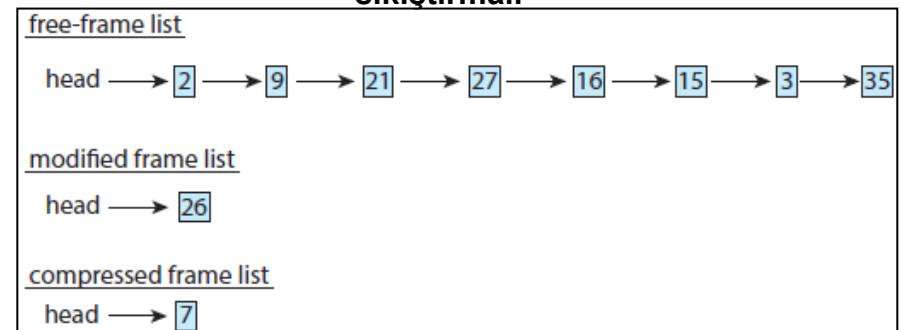
# Memory Compression - Sıkıştırma

- **Bellek sıkıştırma**, değiştirilmiş çerçeveleri swaplamak yerine, birkaç çerçeveyi sıkıştırarak sistemin sayfa değişimini azaltmayı amaçlar.
  - Normal senaryoda, free-frame list 6 çerçeve içersin. Sayfa değiştirmeyi tetikleyen eşik değerin altına düştüğü varsayalım.
    - Algoritma (LRU vb.) free-frame list'e yerleştirmek için 4 çerçeve (15,3,35,26) seçer.
    - Bu çerçeveler modified-frame list'e yerleştirilir ve bu liste swap-out edilir.
      - Böylece çerçeveler free-frame list'e eklenir.
  - Bellek sıkıştırmada ise, free-frame list'ten 7 nolu çerçeve alınır ve 3,15 ve 35 sıkıştırılarak 7 nolu çerçeveye yerleştirilir.
    - Bu 3 çerçeve daha sonra referans edilirse sayfa hatası oluşur ve tekrar yüklenir.

**sıkıştırmasız**



**sıkıştırmalı**



# Memory Compression - Sıkıştırma

- Mobil sistemler genellikle ne sayfa değiştirmeyi ne de takaslamayı desteklemez.
  - Bu nedenle bellek sıkıştırma, Android ve iOS gibi mobil işletim sistemi için bellek yönetimi stratejisinin ayrılmaz bir parçasıdır.
  - Ayrıca, hem Windows10 hem de macOS bellek sıkıştırmasını destekler.
    - Microsoft, Windows 10 çalıştıran cihazlar için ortak bir uygulama platformu sağlayan **Universal Windows Platform** (UWP) mimarisini geliştirdi.
    - macOS, ilk olarak 10.9 sürümü ile bellek sıkıştırmasını destekledi.
      - macOS öncelikle boş bellek yetersiz olduğunda LRU sayfalarını sıkıştırır, sorunu çözmezse sayfa değişimini yapar.
        - Performans testlerinde, bellek sıkıştırmasının SSD ile sayfa değişimlerinden dahi hızlı olduğunu göstermektedir.
  - Veri sıkıştırmada olduğu gibi bellek sıkıştırmada da sıkıştırma algoritmasının hızı ile sıkıştırma oranı önemlidir.
    - Yavaş algoritmalar daha yüksek sıkıştırma oranı sunarken, tersi de mümkündür.
    - Burada hız ve sıkıştırma oranı açısından dengeli algoritmalar tercih edilir.
  - Örn. Microsoft Xpress ve Apple WKdm hızlı ve %30-50 sıkıştırma oranı sunarlar.

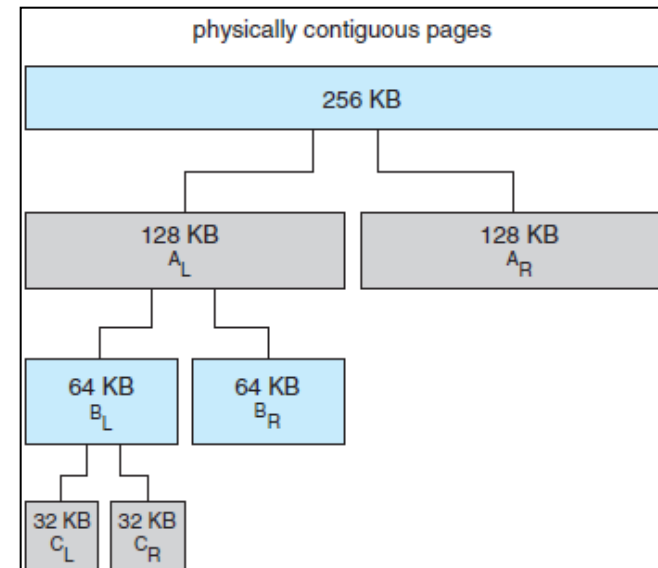


# A. Kernel Mem.-Çekirdek Bellek Tahsisi

- Kullanıcı modunda çalışan bir süreç ek bellek talep ettiğinde, çekirdek tarafından yönetilen free-frame list'ten tahsis edilir.
- Ancak **Kernel Memory** sıradan kullanıcı süreçleri için kullanılan farklı bir bellek havuzundan tahsis edilir. Bunun başlıca iki nedeni vardır:
  1. Çekirdek, bazıları bir sayfadan daha küçük olan çeşitli boyutlardaki veri yapıları için bellek kullanır.
    - Dolayısıyla, çekirdek belleği parçalanma nedeniyle israfı en aza indirecek şekilde kullanır.
    - Bu birçok işletim sistemi çekirdek kodlarını/verilerini sayfalamaya tabi tutmadığı için önemlidir.
  2. Kullanıcı süreçlerine ayrılan sayfaların bitişik fiziksel bellekte olması gerekmez.
    - Ancak belirli donanım sürücüleri, doğrudan fiziksel bellek erişiminde bulunur ve fiziksel olarak belleğin bitişik sayfalarda bulunmasını gerektirebilir.
- Kernel süreçlerine bellek tahsis etmek için kullanılan iki yöntem bulunmaktadır:
  - buddy system, slab allocation

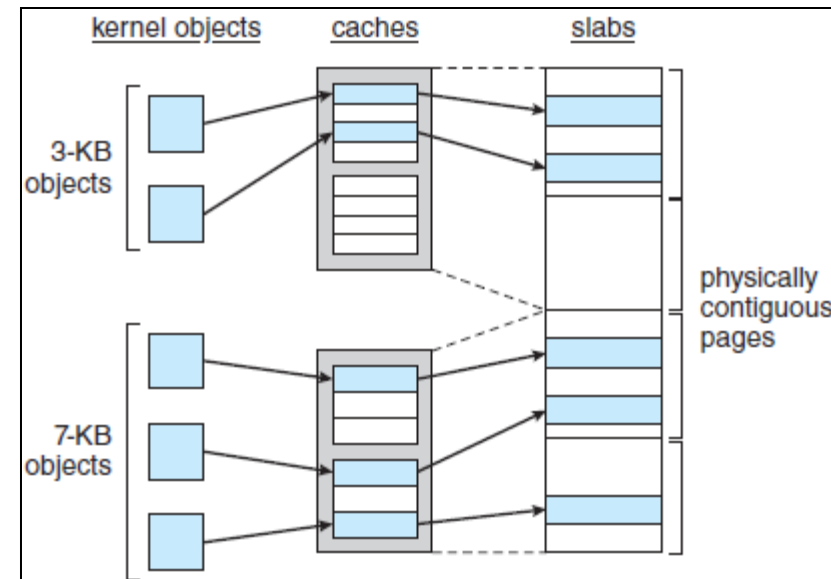
# Allocating Kernel Mem - Buddy System

- Buddy sistemi, fiziksel olarak bitişik sayfalardan oluşan sabit boyutlu segmentlerden bellek tahsis eder.
  - Bu segmentler 2'nin katı (4KB, 8KB, 16KB, vb.) şeklinde boyutlandırılır
  - Örneğin, 11 KB'lık bir istek, 16 KB'lık bir segmentle karşılanır.
  - Örneğin, bir bellek segmentinin 256KB olduğun durumda,
    - Kernel'ın burdan 21 KB'lık bir bellek talep ettiğini varsayalım,
    - Segment başlangıçta 128 KB'lık Buddy'lere bölünür ( $A_L$  ve  $A_R$  adı verelim)
    - Bunlardan biri 64 KB'lık Buddy'lere bölünür ( $B_L$  ve  $B_R$ )
      - 21 KB, 2'nin 5 kuveetine yakın olduğu için (32)
    - Bu nedenle  $B_L$  yada  $B_R$  tekrar iki Buddy'ye bölünür ( $C_L$  ve  $C_R$ )
      - Bu Buddy'lerden birisi 21KB'lık talebi karşılamak için kullanılır.
    - Buddy'ler ihtiyaç duyulursa birleştirilerek daha büyük talepler karşılanabilir
    - En büyük dezavantajı parçalanmanın fazla olmasıdır.
    - Örn. 33 KB'lık bir bellek talep edildiğin 64 KB'lık bir segment tahsis edilmek zorundadır.



# Allocating Kernel Mem - Slab Allocation

- Çekirdek belleğini tahsis için ikinci strateji, **Slab** tahsisi olarak bilinir.
  - Bir slab, fiziksel olarak bitişik bir veya daha fazla sayfadan oluşur.
  - Bir önbellek; bir veya daha fazla slab'dan oluşur,
    - Her çekirdek veri yapısı için bir önbellek vardır.
      - Örn, süreç tanımlayıcıları, dosya nesneleri, semaforlar vb. için ayrı bir önbellek vardır.
      - Her önbellek temsil ettiği çekirdek veri yapılarının örnekleri olan nesneleri barındırır.
      - Şekilde, her biri ayrı önbellekte saklanan iki 3KB'lık ve üç 7KB'lık kernel nesnesi gösterilmiştir.
- Önbellekteki nesnelerin sayısı, ilgili slab'ın boyutuna bağlıdır.
- Örn, 12KB'lık bir slab, (4KB'lık 3 page'den oluşan) 6 adet 2KB'lık nesne depolayabilir.
- Başlangıçta önbellekteki tüm nesneler boş olarak işaretlenir
- Kernel veriyapısı için yeni bir nesneye ihtiyaç duyulduğunda, Slab Tahsis algoritması;
- Önbellekten herhangi bir boş nesne tahsis edilebilir.
- Tahsis edilen nesne kullanılmış olarak işaretlenir.



# Allocating Kernel Mem - Slab Allocation

- Kernel'ın, bir süreç tanımlayıcısını (process descriptor) tutan bir nesne için slab tahsisinden bellek talep ettiği bir senaryoda;
  - Linux'te bir süreç tanımlayıcısı yaklaşık 1,7KB bellek gerektiren *struct task\_struct* tipindedir.
  - Linux kernel'ı yeni bir görev oluşturduğunda, bu nesne için gerekli belleği slab tahsis modülü aracılığıyla önbellekten talep eder.
  - Önbellek isteği, bir slab'a tahsis edilmiş ve free olarak işaretlenmiş bir *struct task\_struct* nesnesini kullanarak yerine getirir.
  - Linux'ta, bir slab üç durumda olabilir:
    1. Dolu: Slab'taki tüm nesneler kullanılmış olarak işaretlenmiş
    2. Boş: Slab'taki tüm nesneler serbest olarak işaretlenmiş
    3. Kısmi: Slab'ta hem kullanılmış hem de serbest nesneler var
  - Slab talepleri öncelikle Kısmi bir slab'dan karşılamaya çalışır, yoksa boş olandan...
  - Boş slab yoksa bitişik fiziksel sayfalardan yeni bir slab ayırır ve önbelleğe atar.

# Allocating Kernel Mem - Slab Allocation

- Slab Allocator, iki temel fayda sağlar.
  1. Parçalanma olmaz. Çünkü her çekirdek veri yapısı için ilişkili bir önbellek vardır.
    - Her önbellek, temsil edilen nesnelerin boyutuna göre parçalara bölünmüş bir veya daha fazla slab'dan oluşur.
    - Bu nedenle, kernel nesneler için bellek talep ettiğinde, slab allocator nesneye gereken bellek miktarını döndürür.
  2. Bellek istekleri hızlı bir şekilde karşılanabilir. Normalde belleği tahsis etme ve serbest bırakma zaman alıcıdır.
    - Ancak, nesneler önceden oluşturulur ve bu nedenle önbellekten hızlı bir şekilde tahsis edilebilir. Ayrıca, çekirdek bir nesneyi serbest bıraktığında, önbelleğe geri döner.
      - Böylece bu nesne çekirdekten gelen sonraki istekler için hemen kullanılabilir.
    - Linux'te sürüm 2.2'ye kadar Buddy Sistem, 2.2'den sonra Slab/SLAB kullanıldı.
    - Linux ayrıca SLAB'ın varyantı olan SLOB ve SLUB'ı da destekler.
      - Gömülü sistemler gibi sınırlı belleğe sahip sistemler için SLOB tasarlanmıştır.
      - 2.6.24'ten sonra SLAB'a göre daha performanslı olan SLUB ayıracısını kullanmaya başladı.

# Diğer Konular - Prepaging

- Bir süreç başlatıldığında saf Demand Paging'de çok sayıda sayfa hatası oluşur.
- **Prepaging (ön sayfalama)**, ihtiyaç duyulacak sayfaların bir kısmını veya tamamını tek seferde önceden belleğe alarak yoğun ilk sayfalama isteğini engellemeye çalışır.
  - **Working-Set modelinde**, her sürecin çalışma kümesindeki sayfaların bir listesi tutulabilir.
    - Bir süreç askıya alınırsa (free-frame nedeniyle), o sürecin çalışma kümesi bilinebilir.
    - Süreç devam ettirildiğinde (I/O bittiğinde veya yeterli boş çerçeve olduğunda), süreci yeniden başlatmadan önce tüm çalışma kümesi belleğe getirilebilir.
  - **Prepaging bazen avantaj sağlasa da; Prepaging'in maliyeti, sayfa hatalarının servis maliyetinden daha az olmalıdır.**
    - Çünkü, prepaging ile belleğe getirilen sayfaların çoğu kullanılmayabilir.
    - Prepaging ile getirilen sayfaların  $\alpha$  kadarının kullanıldığı varsayılırsa ( $0 \leq \alpha \leq 1$ ).
      - $\alpha$  0'a yakınsa, prepaging kaybeder;  $\alpha$  1'e yakınsa, prepaging kazanır.
      - Ayrıca, bir programın prepaging'i zordur. Hangi sayfaların getirilmesi gerektiği belirsizdir.
    - Dosyalara genellikle sırayla erişildiği için bir dosyanın prepaging'i daha kolaydır.
      - Linux'te ***readahead()*** sistem çağrısı ile bir dosyanın içeriği önceden belleğe getirilebilir.

# Diğer Konular – Page Size

- Yeni sistemlerde 2'nin katları şeklinde çeşitli boyutları destekleyen ve genellikle 4.096 ( $2^{12}$ ) ile 4.194.304 ( $2^{22}$ ) bayt arasında değişen sayfa boyutları vardır.
  - Burada sayfa boyunun azaltılması sayfa sayısını artıracığı için page-table'ı artırır.
    - Örn, 4 MB'lık bir sanal bellekte 1KB page-size ile 4096 sayfa, 4KB page-size ile 512 sayfa!
    - Her aktif süreç kendi page-table kopyasına sahip olacağı için büyük page-size istenir.
    - Ancak bellek daha küçük page-size ile daha iyi tahsis edilir. (Dahili parçalanma!)
    - Diğer bir sorun sayfaların HDD'den okunmasıdır. HDD'de I/O zamanı seek, latency ve transfer times olarak hesaplanır. Burada asıl maliyet Seek ve Latency'dir.
      - Seek 5 ms, Latency 3 ms. ve transfer süresi 512 byte için 0.01 ms. (50 MB/sn'lik bir disk için) ise burada büyük page-size disk I/O zamanı açısından daha avantajlı gibi görülmektedir.
    - Ancak küçük page-size locality açısından daha elverişlidir. Daha iyi **resolution** sağlar.
    - Ancak küçük page-size daha fazla sayfa hatasına neden olur.
  - Dolayısıyla doğru page-size nedir? sorusunun doğru bir cevabı yoktur.
    - Dahili parçalanma, locality küçük page-size'ı; page-table, I/O büyük page-size'ı önerir.
    - Ancak mobil sistemlerde dahi page-size'ın daha büyük boyutlarda kullanılma eğilimi vardır.

# Diğer Konular – TLB Erişimi

- TLB isabet oranı (**hit ratio**), page-table yerine TLB'de çözülen sanal adres'in yüzdesine karşılık gelmektedir.
  - İsabet oranını artırmanın yolu TLB girişlerini artırmaktır.
    - Ancak, TLB'de kullanılan bellek hem pahalı hem de çok fazla güç tüketir.
  - İsabet oranıyla ilgili benzer bir metrik **TLB Reach**'dir.
    - TLB Reach, TLB'den erişilebilen bellek miktarını ifade eder. (page-size ile çarpılan kayıt sayısı)
    - İdeal olan bir sürecin working-set'inin TLB'de olmasıdır. Değilse page-table zaman kaybı!
      - TLB'de giriş sayısı artırılırsa TLB Reach'de artar. Ancak yine de yoğun bellek kullanan uygulamalar için working-set'i tutmak için TLB yetmez.
    - TLB Reach'i artırmak için page-size'ı artırmak yada farklı page-size'ları desteklemek düşünülebilir.
      - Page-size'ı artırmak TLB isabet oranını artırsa da dahili parçalanmayı artırır.
        - Büyük page-size gerektirmeyen uygulamalar için gereksiz parçalanmaya neden olur.
      - Çoğu mimari farklı page-size boyutlarını destekler.
        - Önr. Linux'te varsayılan page-size 4 KB iken daha büyük sayfaları da (örn 4 MB) destekler.



# Diğer Konular – TLB Erişimi

- ARMv8 mimarisi farklı boyutlardaki sayfalar ve bölgeler (region) için destek sağlar.
  - Ayrıca, ARMv8'de her TLB girişi bir bitişik bit (contiguous bit) içerir.
  - Bu bit bir TLB girişi için ayarlanmışsa, bu giriş bitişik bellek bloklarını eşler.
  - Bitişik bloklar üç olası durumu, bir TLB girişiyle eşler ve TLB Reach artırılır:
    1.  $16 \times 4$  KB bitişik bloklardan oluşan 64-KB TLB girişi.
    2.  $32 \times 32$  MB bitişik bloklardan oluşan 1 GB TLB girişi.
    3.  $32 \times 64$  KB bitişik bloklardan veya  $128 \times 16$  KB bitişik bloklardan oluşan 2-MB TLB girişi.
- Birden çok sayfa boyutunu desteklemek, TLB'yi yönetmek için donanım yerine işletim sistemi gerektirir.
  - Örn, TLB girişindeki alanlardan birinde frame'in boyutu tutulmalıdır.
  - TLB'yi donanım yerine yazılımda yönetmenin bir maliyeti vardır.
    - Ancak, artan **Hit Ratio** ve **TLB Reach** maliyeti dengeler.

# Diğer Konular – Inverted Page Tables

- IPT sayfa yönetimi modelinin amacı, sanaldan fiziksel adrese çevirileri sağlamak için gereken fiziksel bellek miktarını azaltmaktır.
  - Bu, fiziksel belleğin her çerçevesi için bir girişi olan tablo oluşturularak sağlanır.
    - $\langle \text{process-id, page-number} \rangle$  çifti ile indekslenmiş.
  - Her bir fiziksel çerçevede hangi sanal bellek sayfasının depolandığı tutulur.
    - IPT tabloları bu bilgileri depolamak için gereken fiziksel bellek miktarını azaltır.
    - Ancak, IPT tabloları bir sürecin mantıksal adres alanı hakkında bilgi içermez.
    - Referans edilen bir sayfa bellekte değilse (sayfa hatası) bu bilgi gereklidir.
    - Demand paging, sayfa hatalarını işlemek için bu bilgiye ihtiyaç duyar.
  - Bu bilgi için harici bir page-table (süreç başına) gerekir.
    - Bu tablolar, klasik süreç başına page-table'a benzer ve her sanal sayfanın konumunu tutar.
- Ancak harici page-table'lar, IPT'nin faydasını etkisiz hale getirir mi?
  - Bu tablolara sadece bir sayfa hatası oluştuğunda başvurulduğundan, bellekte tutulmazlar ve gerektiğinde belleğe getirilirler.
    - Ancak harici page-table'da da yeni sayfa hataları olabilir. Önemli gecikmelere neden olur!

# Diğer Konular – Program Structure

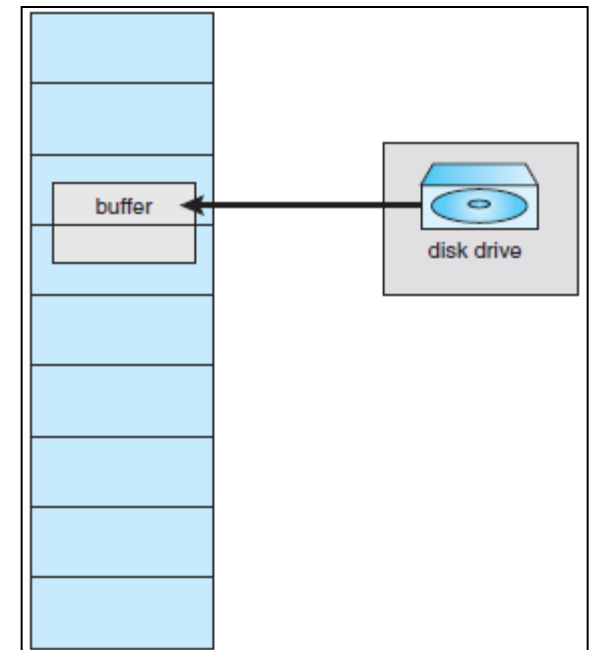
- Demand Paging, programlardan yalıtılmıştır ve kullanıcı belleğin sayfalı yapısından habersizdir.
  - Ancak, kullanıcı (veya derleyici) temeldeki demand paging konusunda farkındalığa sahipse sistem performansı iyileştirebilir.
  - Yapay bir örnek üzerinden; Sayfa boyutunun 256 byte olduğunu varsayalım.
    - 128x128'lik bir dizinin her elemanını 0'layan bir C programı için; (Soldaki kod)
      - Dizi bellekte dizi[0][0], dizi[0][1], dizi[0][2]...dizi[0][127], dizi[1][0], dizi[1][1].. şeklinde tutulur.
      - Yani 256 byte'lık bir page-size için her satır bir sayfa tüketir.
      - Bu kod her sayfa bir elemanı sıfırlar ve bir sonraki sayfa bir sonraki elemanı sıfırlar.
      - İşletim sistemi program 128'den daha az çerçeve ayırırsa  $128 \times 128 = 16,384$  sayfa hatası oluşur.
    - Bunun yerine sağdaki kod, sayfa değiştirmeden önce bir sayfadaki tüm elemanları sıfırlayarak sayfa hata sayısını 128'e düşürür.

```
int i, j;
int[128][128] data;
for (j = 0; j < 128; j++)
  for (i = 0; i < 128; i++)
    data[i][j] = 0;
```

```
int i, j;
int[128][128] data;
for (i = 0; i < 128; i++)
  for (j = 0; j < 128; j++)
    data[i][j] = 0;
```

# Diğer Konular – I/O ve Page Kilitleri

- Demand Paging kullanıldığında, bazen sayfaların bellekte kilitlenmesi gerekir.
  - Kullanıcı belleğine I/O yapıldığında bu durum gerçekleşir.
  - I/O genellikle ayrı bir I/O işlemcisi tarafından uygulanır.
    - Örneğin, bir USB depolama aygıtı için denetleyiciye aktarılacak byte sayısı ve buffer için bir bellek adresi verilir.
    - Transfer tamamlandığında CPU kesmeye uğrar.
  - Bir süreç bir I/O isteği yapar ve I/O kuyruğuna girer.
    - Bu aşamada CPU, diğer süreçlere bırakılır
    - Bu süreçler sayfa hatalarına neden olur.
    - Bu süreçlerden birisi bellek arabelleğini içeren sayfayla değiştirilir.
    - Bir süre sonra, I/O talebi aygıt kuyruğunda işleme alınır ve I/O belirtilen adrese gerçekleşir.
    - Ancak bu çerçeve artık başka bir sürece ait farklı bir sayfa için kullanılmaktadır.



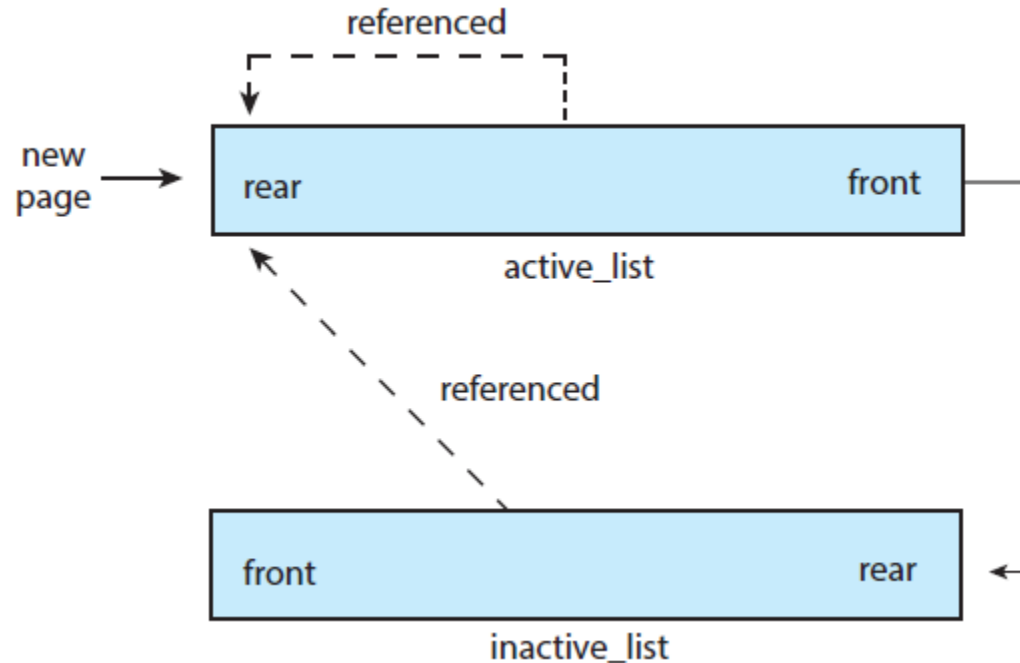
# Diğer Konular – I/O ve Page Kilitleri

- Bu sorunun çözümlerinden biri; asla kullanıcı belleğine I/O uygulamamaktır.
  - Veriler her zaman sistem belleğine yapılır ve kullanıcı belleğine kopyalanır.
    - I/O, yalnızca sistem belleği ile I/O aygıtı arasında gerçekleşir.
    - Bu ekstra kopyalama, kabul edilemeyecek kadar yüksek ek yüke neden olur.
- Diğer çözüm; sayfaların bellekte kilitlenmesidir.
  - Burada, her frame'le bir kilit biti ilişkilendirilir.
    - Frame kilitliyse, sayfa değişimi için seçilemez. I/O tamamlandığında, sayfaların kilidi açılır.
  - Kilit bitleri, işletim sistemi çekirdeğinin bir kısmı veya tamamını kilitlemek için de kullanılır.
  - Kullanıcı süreçleri de sayfaları kilitleyebilir.
    - Örn, veritabanları disk ile bellek buffer'ları arasında veri taşımalarında belleği kilitler.
  - Çoğu işletim sistemi mantıksal adres alanının bir kısmının sabitlenmesi için bir sistem çağrısı sunar.
    - Ancak kötüye kullanılabileceği için bu tür komutlar özel ayrıcalık gerektirir.

# İşletim Sistemi Örnekleri – Linux

- Linux SLAB tahsisi kullanarak kernel belleğini yönetir.
- Linux sanal belleği, sayfaların free-fame list'ten tahsis edildiği Demand Paging kullanır.
- Ayrıca, LRU temelinde Clock Algoritmasına benzer bir sayfa değiştirme algoritması kullanır.
- Linux belleği yönetmek için iki tür sayfa listesi tutar: *active\_list* ve *inactive\_list*.
  - Aktif liste, kullanımda olduğu düşünülen sayfaları içerirken, aktif olmayan liste, yakın zamanda başvurulmamış ve sayfa değişimine uygun sayfaları içerir.
- Her sayfanın referans edildiğinde ayarlanan bir erişim biti (*accessed*) vardır.
  - Bir sayfa referans edildiğinde, erişim biti ayarlanır ve aktif listenin sonuna eklenir.
  - Periyodik olarak aktif listesindeki erişim bitleri sıfırlanır.
  - Zamanla en az kullanılan sayfa aktif listenin en başına geçerek inaktif listeye geçer.
  - İnaktif listedeki bir sayfa referans edildiğinde tekrar aktif listenin sonuna eklenir.

# İşletim Sistemi Örnekleri – Linux



- İki liste görece dengede tutulur ve aktif liste çok fazla büyüdüğünde, aktif listenin başındaki sayfalar, aktif olmayan listeye taşınır.
- Linux çekirdeği, sistemdeki boş bellek miktarını düzenli aralıklarla kontrol eden ve yenileyen bir page-out daemon sürecine (*kswapd*) sahiptir.
  - Boş bellek belirli bir eşiğin altına düşerse, *kswapd* etkin olmayan sayfaları tarar ve bunları free-frame list için page-out yapar.

# İşletim Sistemi Örnekleri – Windows

- Windows 10, Intel (32 ve x86-64) ve ARM mimari sistemleri destekler.
  - 32-bit sistemlerde, bir sürecin varsayılan sanal adres alanı 2 GB'dır, ancak 3 GB'ye kadar genişletilebilir.
  - 32-bit sistemler 4 GB fiziksel belleği destekler.
  - 64 bit sistemlerde Windows 10, 128 TB'lık bir sanal adres alanına sahiptir ve 24 TB'a kadar fiziksel belleği destekler.
- Windows Server sürümleri, 128 TB'a kadar fiziksel belleği destekler.
  - Windows 10, paylaşılan kütüphaneler, demand paging, copy-on-write, paging ve bellek sıkıştırma dahil bellek yönetimi özelliklerinin çoğunu uygular.
  - Windows 10, bellek referanslarının locality'si için sayfa hatalarında yalnızca hataya neden olan sayfayı değil, hataya neden olan sayfadan önce ve sonraki birkaç sayfayı da getiren kümelemeli (**clustering**) demand paging ile sanal belleği taşır.
  - Bir kümenin boyutu sayfa türüne göre değişir.
- Bir veri sayfası için, bir küme üç sayfa içerir (hataya neden olan sayfa, öncesi ve sonrası); diğer tüm sayfa hatalarının küme boyutu yedidir.



# İşletim Sistemi Örnekleri – Windows

- Windows 10'daki sanal bellek yönetiminin önemli bir bileşeni, **working-set**'dir.
- Bir süreç oluşturulduğunda, minimum 50-maksimum 345 sayfalık working-set atanır.
- **Working-set minimum**, sürecin bellekte olması garanti edilen sayfa sayısıdır.
- Yeterli bellek mevcutsa, bir sürece **working-set maximum** kadar bellek atanabilir.
- Bir süreç, **hard working-set limits**'le yapılandırılmadıkça, limitler dikkate alınmaz.
  - Yeterli bellek varsa, bir süreç working-set maximum'un ötesine geçebilir.
  - Benzer şekilde bellek talebinin yüksek olduğu dönemlerde minimumun altına düşebilir.

# İşletim Sistemi Örnekleri – Windows

- Windows, LRU yaklaşımıyla Clock Algoritmasını, local ve global sayfa değiştirme tekniklerinin bir kombinasyonu kullanır.
  - Sanal bellek yöneticisi, boş çerçevelerin bir listesini tutar.
  - Bu listeye ilişkili, yeterli boş belleğin olup olmadığını gösteren bir eşik değeri (**threshold**) vardır.
  - Bellekteki sayfa sayısı «Working-Set Maksimum'un» altında olan bir süreç sayfa hatası alırsa, sanal bellek yöneticisi free-frame list'ten bir sayfa tahsis eder.
  - «Working-Set Maksimum'un» üstünde olan bir süreç sayfa hatası alırsa ve yeterli bellek varsa, sürece boş bir frame tahsis edilir.
- Yani, working-set'in maksimumun ötesine büyümesine izin verir.
  - Ancak boş bellek miktarı yetersizse kernel, local LRU sayfa değiştirme algoritmasını kullanarak, mevcut sürecin working-set'inden sayfa değişimi için bir sayfa seçmelidir
  - Boş bellek miktarı eşik değer altına düştüğünde sanal bellek yöneticisi otomatik working-set düzeltmesi (**automatic working-set trimming**) olarak bilinen teknikle;
- Süreçlere ayrılan sayfaları değerlendirir ve minimum working-set'in üzerinde sayfa tahsis edilen süreçlerden sayfaları kaldırır.

# Bölüm Özeti

- Sanal bellek (**Virtual Memory**), fiziksel belleği büyük bir depolama kümesinde özetler.
- Sanal belleğin faydaları: (1) bir program fiziksel bellekten daha büyük olabilir, (2) bir programın tamamen bellekte olması gerekmez, (3) süreçler belleği paylaşabilir ve (4) süreçler daha verimli oluşturulabilir.
- Sayfalama isteği (**demand paging**), sayfaların yalnızca program yürütülürken talep edildiğinde yüklenmesidir. Talep edilmeyen sayfalar asla belleğe yüklenmezler.
- Hafızada olmayan bir sayfa referans edildiğinde sayfa hatası (**page-fault**) oluşur. Sayfanın, yardımcı bellekten ana bellekte uygun/boş bir çerçeveye getirilmesi gerekir.
- **Copy-on-write**, bir child sürecin parent süreçle aynı adres alanını paylaşmasına izin verir. Child veya parent süreç sayfaya yazarsa (değiştirirse), sayfanın bir kopyası oluşturulur.
- Kullanılabilir bellek azaldığında, sayfa değiştirme (**page-replacement**) algoritması, yeni sayfa için ana bellekteki mevcut bir sayfayı seçer. Sayfa değiştirme algoritmaları arasında **FIFO**, **Optimal** ve **LRU** bulunur. Saf LRU algoritmasının uygulanması pratik değildir. Çoğu sistem LRU temelli yaklaşım (**LRU-approximation**) algoritmalarını kullanır.

# Bölüm Özeti

- Global sayfa değiştirme (**global page-replacement**) algoritmaları, sayfa değişimi için sistemdeki herhangi bir süreçten sayfa seçerken, yerel sayfa değiştirme (**local page-replacement**) algoritmaları, sayfa hatası oluşan süreçten sayfa seçer.
- Bir sistem, sayfalamaya yürütümden fazla efor harcadığında çökme (**thrashing**) oluşur.
- Yerellik (**locality**), aktif olarak birlikte kullanılan bir dizi sayfayı ifade eder. Çalışma kümesi (**working-set**) locality'ye dayanır ve sürecin o an kullandığı sayfalar kümesidir.
- Bellek sıkıştırma (**memory compression**), birkaç sayfayı tek bir sayfada sıkıştıran bir bellek yönetim tekniğidir. Sayfalamaya alternatiftir ve sayfalamayı desteklemeyen mobil sistemlerde kullanılır.
- Çekirdek belleği (**kernel memory**), kullanıcı süreçlerinden farklı tahsis edilir; çeşitli boyutlarda bitişik parçalar. Kullanılan teknikler: (1) buddy system, (2) slab allocation'dir
- TLB erişimi (**TLB reach**), TLB üzerinden erişilebilen bellek miktarını ifade eder ve TLB'deki kayıt sayısının sayfa boyutuyla çarpımına eşittir. TLB erişimini artırmanın bir yolu, sayfa boyutunu (**page-size**) artırmaktır.
- Linux, Windows ve Solaris, diğer bir takım özelliklerle birlikte benzer şekilde **demand paging** ve **copy-on-write** kullanarak sanal belleği yönetir. Her sistem ayrıca **Clock algoritması** olarak bilinen bir LRU temelli algoritmayı kullanır.

# Çalışma Soruları

- 16 bit mantıksal ve fiziksel adres uzayına sahip bir sistemin page-size'ı 4096 Byte'dır.
- Yan tarafta bu sistemde yer alan bir page-table örneği verilmiştir. Bu tabloya göre bir page'in referans bit'i sayfa referans edildiğinde «1»'e kurulur. Bir page için page-frame değeri «-» ise sayfanın bellekte olmadığını ifade eder. Sayfa değiştirme algoritması olarak yerel LRU kullanılır. Buna göre;
  - a. Aşağıdaki mantıksal adresleri (onaltılık) fiziksel adreslere dönüştürün. Page-table'da referans bitini ayarlayın. • 0x621C • 0xF0A3 • 0xBC1A • 0x5BAA • 0x0BA1
  - b. Page-fault'a neden olan bir mantıksal adres örneği (onaltılık) yazın.
  - c. LRU sayfa değiştirme algoritması, bir page-fault oluştuğunda hangi sayfaları seçer?

| Page | Page Frame | Reference Bit |
|------|------------|---------------|
| 0    | 9          | 0             |
| 1    | -          | 0             |
| 2    | 10         | 0             |
| 3    | 15         | 0             |
| 4    | 6          | 0             |
| 5    | 13         | 0             |
| 6    | 8          | 0             |
| 7    | 12         | 0             |
| 8    | 7          | 0             |
| 9    | -          | 0             |
| 10   | 5          | 0             |
| 11   | 4          | 0             |
| 12   | 1          | 0             |
| 13   | 0          | 0             |
| 14   | -          | 0             |
| 15   | 2          | 0             |

# Çalışma Soruları

- Aşağıdaki page-referans dizesini 3 çerçeveli bir demand paging uygulayan bir sistemde FIFO, LRU ve OPT algoritmaları için uygulayın. Her algoritma için oluşan sayfa hatası sayılarını bulun. Başlangıçta frame'lerin boş olduğunu kabul edin.
- 2, 6, 9, 2, 4, 2, 1, 7, 3, 0, 5, 2, 1, 2, 9, 5, 7, 3, 8, 5

|            |   |   |   |   |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |                 |
|------------|---|---|---|---|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|-----------------|
| FIFO       | 2 | 6 | 9 | 2 | 4        | 2        | 1        | 7        | 3        | 0        | 5        | 2        | 1        | 2        | 9        | 5        | 7        | 3        | 8        | 5        | 18 sayfa hatası |
| Çerçeveler | 2 | 6 | 9 |   | 4        | 2        | 1        | 7        | 3        | 0        | 5        | 2        | 1        |          | 9        | 5        | 7        | 3        | 8        | 5        |                 |
|            |   | 2 | 6 |   | 9        | 4        | 2        | 1        | 7        | 3        | 0        | 5        | 2        |          | 1        | 9        | 5        | 7        | 3        | 8        |                 |
|            |   |   | 2 |   | 6        | 9        | 4        | 2        | 1        | 7        | 3        | 0        | 5        |          | 2        | 1        | 9        | 5        | 7        | 3        |                 |
|            |   |   |   |   | <u>2</u> | <u>6</u> | <u>9</u> | <u>4</u> | <u>2</u> | <u>1</u> | <u>7</u> | <u>3</u> | <u>0</u> |          | <u>5</u> | <u>2</u> | <u>1</u> | <u>9</u> | <u>5</u> | <u>7</u> |                 |
| LRU        | 2 | 6 | 9 | 2 | 4        | 2        | 1        | 7        | 3        | 0        | 5        | 2        | 1        | 2        | 9        | 5        | 7        | 3        | 8        | 5        | 17 sayfa hatası |
| Çerçeveler | 2 | 6 | 9 | 2 | 4        | 2        | 1        | 7        | 3        | 0        | 5        | 2        | 1        | 2        | 9        | 5        | 7        | 3        | 8        | 5        |                 |
|            |   | 2 | 6 | 9 | 2        | 4        | 2        | 1        | 7        | 3        | 0        | 5        | 2        | 1        | 2        | 9        | 5        | 7        | 3        | 8        |                 |
|            |   |   | 2 | 6 | 9        | 9        | 4        | 2        | 1        | 7        | 3        | 0        | 5        | 5        | 1        | 2        | 9        | 5        | 7        | 3        |                 |
|            |   |   |   |   | <u>6</u> |          | <u>9</u> | <u>4</u> | <u>2</u> | <u>1</u> | <u>7</u> | <u>3</u> | <u>0</u> |          | <u>5</u> | <u>1</u> | <u>2</u> | <u>9</u> | <u>5</u> | <u>7</u> |                 |
| OPT        | 2 | 6 | 9 | 2 | 4        | 2        | 1        | 7        | 3        | 0        | 5        | 2        | 1        | 2        | 9        | 5        | 7        | 3        | 8        | 5        | 13 sayfa hatası |
| Çerçeveler | 2 | 6 | 9 | 9 | 4        | 4        | 1        | 7        | 3        | 0        | 5        | 5        | 5        | 5        | 9        | 9        | 7        | 3        | 8        | 8        |                 |
|            |   | 2 | 6 | 6 | 9        | 9        | 9        | 1        | 1        | 1        | 1        | 1        | 1        | 1        | 5        | 5        | 9        | 7        | 3        | 3        |                 |
|            |   |   | 2 | 2 | 2        | 2        | 2        | 2        | 2        | 2        | 2        | 2        | 2        | 2        | 1        | 1        | 5        | 5        | 5        | 5        |                 |
|            |   |   |   |   | <u>6</u> |          | <u>4</u> | <u>9</u> | <u>7</u> | <u>3</u> |          |          |          | <u>2</u> |          | <u>1</u> | <u>9</u> | <u>7</u> |          |          |                 |

# Çalışma Soruları

- Aşağıdaki page-referans dizelerini 3 çerçeveli bir demand paging uygulayan bir sistemde FIFO, LRU ve OPT algoritmaları için uygulayın. Her algoritma için oluşan sayfa hatası sayılarını bulun. Başlangıçta frame'lerin boş olduğunu kabul edin.
  - a. 0, 6, 3, 0, 2, 6, 3, 5, 2, 4, 1, 3, 0, 6, 1, 4, 2, 3, 5, 7
  - b. 3, 1, 4, 2, 5, 4, 1, 3, 5, 2, 0, 1, 1, 0, 2, 3, 4, 5, 0, 1
  - c. 4, 2, 1, 7, 9, 8, 3, 5, 2, 6, 8, 1, 0, 7, 2, 4, 1, 3, 5, 8
  - d. 0, 1, 2, 3, 4, 4, 3, 2, 1, 0, 0, 1, 2, 3, 4, 4, 3, 2, 1, 0